

Simple and Efficient Randomized Wait-Free Locks

Kahbod Aeini, Dante Bencivenga, George Giakkoupis, Philipp Woelfel

▶ **To cite this version:**

Kahbod Aeini, Dante Bencivenga, George Giakkoupis, Philipp Woelfel. Simple and Efficient Randomized Wait-Free Locks. PODC 2026 - 45th ACM Symposium on Principles of Distributed Computing, Jul 2026, Egham, United Kingdom. ⟨10.1145/3796701.3815952⟩. ⟨hal-05628351⟩

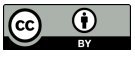
HAL Id: hal-05628351

https://inria.hal.science/hal-05628351v1

Submitted on 20 May 2026

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



Distributed under a Creative Commons CC BY 4.0 - Attribution - International License

Simple and Efficient Randomized Wait-Free Locks

KAHBOD AEINI, University of Calgary, Canada
DANTE BENCIVENGA, University of Calgary, Canada
GEORGE GIAKKOUPIS, Inria, University of Rennes, CNRS, IRISA, France
PHILIPP WOELFEL, University of Calgary, Canada

We present randomized wait-free lock implementations that are simple and time- and space-efficient. One of them uses only three shared variables and has expected step complexity $O(\kappa \log^2 \kappa)$, where κ is the maximum point contention. The other ones have optimal expected step complexity $O(\kappa)$, but require $O(\log n)$ space, where n is the number of processes in the system. All of our algorithms can be easily implemented on standard hardware that supports compare-and-swap and fetch-and-increment/decrement operations.

1 Introduction

Locking is one of the most fundamental and important techniques for concurrent programming. Developers use locks to protect shared memory variables and data structures from concurrent access, thus essentially forcing executions on those protected resources to be sequential. This permits simple algorithm design, and correctness can be reasoned about under assumption of sequentiality.

Classical locks [10, 17, 19] are *blocking*, because processes wanting to access the protected resource must wait until the lock is free. This can cause undesirable delays; a single slow or faulty process can slow down or halt all other participating processes.

In order to avoid this disadvantage of locking, one can try to design *lock-free* algorithms, which guarantee that always *some* process makes progress. But these algorithms are generally more complicated, which may cause decreased performance and implementation errors. Moreover, reasoning about correctness is often challenging for such intricate designs. Ideally, one would aim for algorithms that are *wait-free*, which guarantee that *every* process makes progress, provided that it takes sufficiently many steps. But wait-freedom is even harder to achieve than lock-freedom, and wait-free solutions are often too complicated to be considered practical [14, 18, 21]. With randomized algorithms, we can instead achieve the slightly weaker *randomized wait-freedom*, which guarantees that every process's expected number of steps is finite, given a particular adversarial scheduler [3, 11].

Turek, Shasha and Prakash [22] and, independently, Barnes [4] suggested a general method for concurrent programming, that combines the benefits of locks and lock-freedom. Ben-David, Blelloch, and Wei [6] extended and formalized this approach, calling it *lock-free locks*.

In their framework, one assumes that each participating process aims to protect a piece of code, called a *thunk* [15], which it wants to execute in isolation. This can obviously be achieved by using a classical lock, at the expense of blocking code. The lock-free lock approach avoids blocking, and instead uses helping: When a process gets the lock for the purpose of executing a thunk, other processes help to execute that thunk instead of just waiting.

Helping is facilitated by the use of *idempotent thunks* [4–6, 22, 23]. Idempotence [7–9] means, roughly, that if the same thunk is executed multiple times (possibly concurrently), then the result is no different from it being executed exactly once. Recent work [5, 6] shows how to simulate any thunk that uses only read, write, and compare-and-swap (CAS) operations with idempotent code

Authors' Contact Information: Kahbod Aeini, University of Calgary, Calgary, Canada, kahbod.aeini@ucalgary.ca; Dante Bencivenga, University of Calgary, Calgary, Canada, drbenciv@ucalgary.ca; George Giakkoupis, Inria, University of Rennes, CNRS, IRISA, Rennes, France, george.giakkoupis@inria.fr; Philipp Woelfel, University of Calgary, Calgary, Canada, woelfel@ucalgary.ca.

using the same set of operations. (CAS is a commonly available and widely used synchronization primitive. A CAS(old, new) operation atomically compares the value of a shared memory variable with old, and changes it to new, if and only if there is a match.) The simulation is efficient, and does not increase the asymptotic step complexity of the underlying thunk.

To execute an idempotent thunk in a protected manner, a process creates a *descriptor*, which stores the thunk's code. It then calls TryLock(d), where d is a reference to that descriptor. Multiple concurrent TryLock() calls agree on one of the descriptors that were used as arguments, and execute the chosen descriptor's thunk. In a Lock(d) call, a process can then repeatedly call TryLock(d), until its thunk has been chosen and executed.

As observed by Ben-David, Blelloch, and Wei [6], TryLock() can easily be implemented with a single shared memory variable that stores the address of the currently chosen descriptor. In a TryLock(d) method, a process can simply first execute the thunk of the descriptor pointed to by the variable, and then try to change the variable's value to d, using a CAS operation.

This simple solution is appealing, because it uses minimal (constant) space overhead and has low amortized step complexity. Specifically, if the *maximum point contention* is κ (i.e., at any point there are at most κ TryLock() calls in progress), and thunks have maximum step complexity T, then the amortized step complexity is $O(\kappa \cdot T)$, which is asymptotically optimal. However, this solution is only lock-free, and thus can potentially prevent some processes from performing their desired thunks indefinitely (starvation).

This can be avoided with (randomized) *wait-free* locks. These can be obtained, for example, with universal constructions [2, 14]. These are generic methods that can be used to transform any sequential specification into a concurrent algorithm. But most of those yield high step and space complexity. In a system with n processes, typically each process must help $\Omega(n)$ other thunks before its own thunk gets executed, even if the number of competing processes is small. A universal construction of Afek, Dauber, and Touitou [2] yields wait-free locks, where each process needs to help only $O(\kappa)$ other thunks and perform $O(\kappa)$ other steps, before its own thunk gets executed. While this step complexity is asymptotically optimal, the solution requires $\Theta(n)$ shared memory variables. Ben-David and Blelloch [5] recently proposed a randomized solution for implementing a set of multiple wait-free locks. Each thunk is associated with up to L locks, and it will be executed only when it has been chosen on all of them. If the maximum point contention on each lock is κ, then each process needs to help $O(\kappa^3 L^3)$ other thunks in expectation, before its own thunk gets executed. Thus, applied to a single lock, a process may need to help $O(\kappa^3)$ thunks. The authors do not analyze the space complexity, but it seems to require $\Omega(n^2)$ memory variables for each lock.

Thus, all existing (randomized) wait-free solutions have a space overhead of at least $\Omega(n)$ variables per lock, and only the solution of Afek, Dauber, and Touitou [2] achieves optimal step complexity.

In this paper we propose randomized wait-free locks that exhibit either optimal (i.e., constant) space overhead and near-optimal expected step complexity, or achieve optimal expected step complexity using $O(\log n)$ additional space. All of them can be implemented with standard hardware operations, namely read, write, compare-and-swap (CAS), and fetch-and-increment/decrement (FAID).

In our algorithms, the total number of thunks a process has to execute (help) during its attempt to get the lock is asymptotically the same as the number of other steps it takes. Therefore, when discussing step complexity, we simply state how many *non-thunk steps* a process takes.

Our first algorithm, a preliminary version of which appeared in [1], is very simple and extremely easy to implement. It uses one CAS object, one FAID object, and one register. In a single TryLock() call, a process helps at most two other thunks, and the call succeeds with probability at least

Ref.	Semantics	Space	Step Comp.	Det./Rand.	Adversary
[2]	non-abortable lock	$\Theta(n)$	$O(\kappa)$	deterministic	—
[5]	TryLock	$\Theta(n^2)$	$O(\kappa^3)$	randomized	oblivious
Algorithm 1	TryLock	3	$O(\kappa \log^2 \kappa)$	randomized	weak adaptive
Algorithm 2	abortable lock	$\Theta(\log n)$	$O(\kappa)$	randomized	oblivious
Algorithm 3	non-abortable lock	$\Theta(\log n)$	$O(\kappa)$	randomized	weak adaptive

Table 1. Comparison of wait-free locks that adapt to point contention. Space refers to the number of shared variables used. Step complexity refers to the number (or expected number if the algorithm is randomized) of non-thunk steps a process has to perform in a single `Lock`(ω) call, or in multiple `TryLock`(ω) calls, until the thunk of ω has been executed. The algorithm in [2] is a universal construction, but can be easily turned into a wait-free lock. The focus of [5] is on providing multiple locks, and thus the algorithm is not optimized for a single lock.

$\Omega(1/(1 + k \log^2 k))$, where k is the point contention at the beginning of the `TryLock()` call. Thus, in order for its own thunk to be executed a process needs to perform in expectation $O(1 + \kappa \log^2 \kappa)$ non-thunk steps, where κ is the maximum point contention. Contrary to all other (randomized) wait-free locks we know of (and also most blocking locks), the number of processes in the system can be arbitrarily large. (This assumes that the FAID object has sufficiently many bits to keep track of the number of concurrent `TryLock()` calls, which is usually not a problem.) Moreover, it works against a (weak) adaptive adversary that can make its scheduling decisions in full knowledge of the system's state. The only restriction is that whenever a process makes a random decision, then it also executes its next shared memory step immediately after (i.e., no other processes may be scheduled to take steps in-between).

The key difference between `TryLock`(d) and `Lock`(d) is flexibility. With `TryLock`(d), after each failed attempt, a process can either try again (possibly after performing other work) or abandon its effort to execute d's thunk. In contrast, `Lock`(d) is more restrictive: once the method has been invoked, the calling process must continue contributing work to the method call until the thunk has been executed.

An abortable lock supports only the `Lock`(d) method but offers some of the flexibility of `TryLock`(d). Specifically, it allows the process to abort its attempt to execute d's thunk at any time. When a process chooses (or is instructed by an adversary) to abort, the `Lock`(d) call terminates within a constant number of steps in the worst-case. The thunk for d may or may not execute, and a return value indicates the outcome.

Our second algorithm provides such an abortable `Lock()` implementation. It improves on the time complexity relative to repeated applications of the first algorithm. This comes at the cost of a slightly increased space complexity of $O(\log n)$, where n is the number of processes in the system (which must now be known by the algorithm). In a `Lock()` call, a process repeatedly tries to have its thunk executed until either it succeeds or the process receives an abort signal; each such attempt involves executing at most a constant number of thunks. A process needs to perform in expectation $O(\kappa)$ non-thunk steps before its own thunk is executed, where κ is the maximum point contention of the `Lock()` call. This is asymptotically optimal. Unlike the first algorithm, this one works only against an oblivious adversary, which chooses the order in which processes take steps independently of processes' random choices.

Our last algorithm achieves optimal expected step complexity and uses logarithmic space, similar to the second algorithm. However, it works against the same adaptive adversary that our first algorithm works against. A disadvantage is that it only implements a non-abortable lock.

To the best of our knowledge, our three algorithms are the first randomized wait-free solutions to locking that combine sublinear space complexity and optimal or almost optimal expected step complexity. Moreover, especially our first algorithm is very simple. All algorithms can easily be implemented from shared memory primitives that are available on common hardware. We do not make any unrealistic assumptions about the size of our shared variables: On a typical 64-bit architecture, where only 45 bits are available for addressing heap-allocated memory [16], the number of supported concurrent `TryLock()` or `Lock()` calls is only limited by the number of distinct descriptors that can be allocated. However, we assume that it is the user's responsibility to allocate descriptors, and reclaim their memory safely.

2 Preliminaries

2.1 Shared Memory Model

We assume that a number of processes execute their programs asynchronously and communicate only via atomic operations on shared objects. Our first algorithm works for an arbitrary (unknown) number of processes. For the other two algorithms, we assume a global upper bound, n, on the number of processes, and the implementation uses n. An *execution* consists of a series of shared memory steps, each of which is one atomic operation on one shared memory object. Each such step occurs at an integer point in time $t \geq 1$ of the execution, such that exactly one shared memory step occurs at each point.

We consider three types of shared memory objects: registers, fetch-and-increment/decrement (FAID) objects, and compare-and-swap (CAS) objects.

A FAID object holds an integer and supports the atomic operations `FAI()`, `FAD()`, and `Read()`. A `FAI()` operation increases the value of the integer by one, and returns its value from before the increment. A `FAD()` operation decreases the value of the integer by one, and returns its value from before the decrement. A `Read()` operation returns the object's current value.

A CAS object holds a value and supports the atomic operations `CAS()` and `Read()`. A `CAS(`*old*, *new*`)` operation is successful and changes the object's value to *new* provided it is equal to *old* immediately before the operation. Otherwise, the operation fails and has no effect. Often, `CAS()` operations are assumed to return a Boolean indicating success or failure, but our algorithms do not require this. A `Read()` operation returns the object's value.

Processes are said to *invoke* and *respond* from a method call, when they perform the first and last shared memory steps of the call, respectively. A method call is *pending* at point t if it has been invoked but has not yet responded by point t. A process p is *active* at point t if there is a pending method call at point t invoked by p. The *point contention* at point t is defined as the number of active processes at point t. We say that a method call *completes* in an interval I if it both invokes and responds within I.

2.2 Wait-Free Locks

We follow the approach of Ben-David and Blelloch [5] in defining (randomized) wait-free locks, thunks, and related concepts (see their paper for more precise definitions). A *thunk* is a procedure with no arguments [15], although it is possible to wrap its inputs in a closure [20], and they can emulate return values by writing to a specified location in shared memory.

A descriptor ω stores a thunk $\omega.T$ and possibly some additional auxiliary variables. It provides two methods, `execute()` and `isDone()`. Method ω.`execute()` executes ω's thunk, and method ω.`isDone()` returns a Boolean value, which is `True` if and only if ω.`isDone()` is invoked after some ω.`execute()` call has responded. (It is trivial to implement these methods using a Boolean register, which is initially `False`, and which is set to `True` in the last step of an `execute()` method.) If a

process completes an ω.execute() call at or before point t, then we also say that ω is *done* at point t (which implies that any invocation of ω.isDone() at point t returns True).

A *wait-free try-lock* object provides a method TryLock(ω), which guarantees that either ω's thunk completes within the TryLock() call (the call succeeds), or that the given thunk is not invoked during the call (the call fails). A Boolean return value indicates success or failure.

A *wait-free lock* object provides a method Lock(ω). This object can be abortable or non-abortable. In the abortable case, a process that executes Lock(ω) may be signalled (by the system, at any adversarially chosen point in time) that it is required to finish its Lock() call within a constant number of steps (in the worst-case). To that end, we assume that each process has read-only access to a variable abort_sig, which is False whenever the process invokes a Lock() method, and whose value can change to True (but not back to False) at any point in time during that Lock() call. We say the process receives the abort signal when that happens. A process's Lock(ω) call is allowed to fail (meaning $\omega.T$ does not get executed), if the process receives the abort signal during that call. A Boolean return value indicates whether the Lock() call was successful or not. In a non-abortable lock processes are not being signalled, and thus Lock() calls are always successful (and do not return anything). In either case, if a Lock(ω) call is successful, then the thunk of ω is completed during the execution of the method call.

We require that no two descriptors store the same thunk, and no two Lock() calls can use the same descriptor. We do allow TryLock(ω) to be repeatedly called for the same ω, but only if the calling process is the same for each call, and each of its prior TryLock(ω) calls responded with False. This is a common assumption for similar algorithms using descriptors (see, e.g., [13] Sections 4.1 and 6.1, and [5] Section 7).

Given a thunk ω, we say that $\omega.T$'s *execution interval* begins with the first invocation of any ω.execute() call, and ends with the response of the first ω.execute() call if it exists, and with the end of the execution otherwise.

All wait-free lock and try-lock objects in this paper satisfy the following two safety properties:

(S1) (Mutual exclusion) For any two distinct thunks $T_1 \neq T_2$ that are executed, their execution intervals are disjoint.

(S2) For any descriptor ω, if thunk $\omega.T$ is executed, then its execution interval is within a single TryLock(ω) or Lock(ω) call. Moreover, a TryLock(ω) call and the Lock(ω) call of an abortable lock returns True, if and only if it responds after $\omega.T$ has been executed.

2.3 Adversaries

Adversaries are used to model the influence of process's random decisions on the order in which processes take steps. An adversary determines at any given point in the execution the process who takes the next step. If the process has no pending next step, then the adversary can also make it invoke a TryLock() or Lock() call, whose descriptor satisfies the requirements specified above. For our abortable lock in Algorithm 2, the adversary can also send the abort signal to any process that has a pending Lock() call, by setting its abort_sig variable to True.

For Algorithms 1 and 3 we assume an adaptive adversary model. This adversary schedules the next process based on the entire system state. When an active process is scheduled, that process immediately performs all its non-shared memory steps including coin flips, followed by a single shared memory step. This is often called the *weak* adaptive adversary, distinguishing it from the *strong* adaptive adversary, which can schedule processes to perform coin flips, without having them also execute their next shared memory step [12].

The result of our Algorithm 2 is stated for the oblivious adversary, meaning that scheduling decisions are made independently of the outcomes of coin flips. That is, the order in which processes

Shared Data:

- FAID C
- Register R
- CAS object L

Initialization:

- $C \leftarrow 0$
- R and L reference a dummy descriptor with isDone() = True

HelpAndPromote()

```
1  w ← L.Read()
2  w.execute()
3  d ← R.Read()
4  L.CAS(w, d)
```

TryLock(descriptor D)

```
   // For each descriptor there is a unique process that can call TryLock() on it,
      if and only if no prior TryLock() call for the same descriptor returned True
5  b ← C.FAI() + 1
6  with probability 1/(7b log²(b+1)) do R.Write(D)
7  for i ∈ {1, 2} do HelpAndPromote()
8  if L.Read() = D then D.execute()
9  C.FAD()
10 return D.isDone()
```

Algorithm 1. Constant-space near-linear-time TryLock algorithm for the weak adaptive adversary

take steps is fixed in advance. However, our analysis considers a stronger, non-oblivious adversary, whose behavior is defined in an ad-hoc manner tailored specifically to our algorithm.

3 Algorithms

In this section we present our three algorithms in detail. Analyses and correctness proofs follow in Section 4.

3.1 Optimal-Space TryLock for Adaptive Adversary

Our first algorithm, presented in Algorithm 1, is based on the simple idea that processes randomly choose to write or not write to a shared variable R. The probability of writing should be slightly smaller than the inverse of the contention. As a result, if a process decides to write, then it is unlikely to be overwritten by any "competing" process, because the more processes are competing, the smaller their probability is to also write.

To measure contention, we employ a FAID variable C, which a process increments at the beginning of its TryLock() call and decrements at the end of it. We say a process *observes* contention k, if $k-1$ is the value returned by the C.FAI() operation—thus, processes set their local variable b to their observed contention k in line 5. Moreover, we use a CAS object L that stores a reference to a descriptor of the thunk that can currently be executed. Initially, L references a dummy descriptor whose thunk does nothing and for which isDone() always returns True.

In a TryLock(D) call, after incrementing C in line 5 and observing contention b, the calling process p randomly decides to write to a shared variable R, or to not write (line 6). The probability that p writes is $1/(7b\log^2(b+1))$, with the constant factor $1/7$ carefully chosen to achieve our desired bounds. The process then performs two calls to a helper method, HelpAndPromote(), in line 7. In each execution of that method, p first reads the currently chosen descriptor w from L (in line 1), and then executes w's thunk in line 2. The process then reads a descriptor d from R in line 3, and then tries to CAS d into L in line 4. We say d gets *promoted* if the CAS is successful.

The crucial property of HelpAndPromote() is the following. If process p chooses to write its descriptor D_p to R in line 6, and then two complete HelpAndPromote() calls are executed before any process overwrites D_p from R, then D_p gets promoted by the time the second HelpAndPromote() terminates (see Lemma 4.5). This property also ensures that a process that executes line 6 while $R = D_p$ cannot complete its TryLock() call before either D_p gets promoted or overwritten. As a result, if the adversary schedules multiple processes to perform that line after p has written D_p to R, then these processes can be ordered such that the i-th one observes contention at least i. This lets us conclude that the probability that any such process can overwrite D_p from R before D_p gets promoted, is at most $O\left(\sum_{i=1}^{\infty} 1/(i\log^2(i+1))\right)$. That series converges, and so by choosing $1/7$ as the constant factor, we can make that probability less than $1/2$.

It follows that by the time process p's execution reaches line 8, D_p has been promoted with asymptotically the same probability with which it wrote to R in line 6, i.e., $\Omega(1/(b\log^2(b+1)))$. In line 8, p then reads L, and if it finds its own thunk there, then it executes it. This guarantees that if D_p gets promoted prior to that point, then D_p's thunk will also get executed. Finally, in line 9 p decrements C, and then in line 10 it determines whether its thunk has been executed, and returns the appropriate result.

We analyze this algorithm and prove its correctness in Section 4.1. In particular, Theorem 4.9 states that each TryLock() call succeeds with probability $\Omega\left(\frac{1}{k\log^2(k+1)}\right)$, where k is the contention at the invocation of the call.

3.2 Optimal-Time Abortable Lock for Oblivious Adversary

The pseudocode of the algorithm is given in Algorithm 2. Similarly to Algorithm 1, it uses a FAID register C and a CAS object L, but here R is an array of $\Theta(\log n)$ registers instead of a single register. The shared variables in the array R serve similar purposes to the single register R in Algorithm 1. Another difference is that the descriptors used are equipped with an additional CAS field, stat, which takes values from set {wait, exec, susp}. The descriptor D passed as an argument to a Lock() call is required to have D.stat = wait initially, which indicates its thunk is not yet ready to be executed. During the call, D.stat changes once to either susp, if the invoking process p sees an abort signal, or to exec, if D is selected to be executed next.

The core idea to achieve optimal time for this algorithm lies in how the indices in R are chosen to be written to and read from, when deciding the next descriptor to promote. Processes choose an index in R to write to according to a geometric distribution (in line 30). The index of R which is read to select the next descriptor to be promoted is chosen based on the logarithm of the current contention (in line 16). Since the contention is (within constant factors) inversely proportional to the probability of writing to the chosen index, an expected constant number of processes succeed in writing their descriptor to the index read in line 17. Optimal time then follows from each process having a fair chance at having their descriptor promoted next. We explain this more formally at the end of this section.

A Lock(D) call invoked by process p starts by incrementing FAID object C in line 23 (ignore line 22 for now), and ends by decrementing C in line 33, before returning in line 34, so that C

Shared Data:

- FAID C
- Registers $R[j]$ for $j \in \{0, \ldots, \lfloor \log n \rfloor + 1\}$
- CAS object L

Initialization:

- $C \leftarrow 0$
- $R[j]$ for $0 \le j \le \log n$, and L reference a dummy descriptor with isDone() = True and stat = exec
- ω.stat $\leftarrow$ wait, for any new descriptor ω

HelpAndPromote(descriptor D)

```
11 w ← L.Read()
12 w.stat.CAS(wait, exec)
13 if w.stat.Read() = exec then
14     w.execute()
15 s ← D.stat.Read()
16 λ ← ⌊log C.Read()⌋
17 d ← R[λ].Read()
18 if d.stat.Read() = wait then w′ ← d
19 else if s = wait then w′ ← D
20 else w′ ← w
21 L.CAS(w, w′)
```

Lock(descriptor D)

```
   // each Lock() call takes as argument a distinct descriptor pointing to a
      distinct thunk
22 for i ∈ {1, 2} do HelpAndPromote(D)
23 C.FAI()
24 HelpAndPromote(D)
25 while D.isDone() = False and D.stat.Read() ≠ susp do
26     if abort_sig then
27         D.stat.CAS(wait, susp)
28         HelpAndPromote(D)
29     else
30         g ← random sample from distribution Pr[g = j] = { 2^{-j-1},  0 ≤ j ≤ ⌊log n⌋
                                                           { 2^{-j},    j = ⌊log n⌋ + 1
31         R[g].Write(D)
32         for i ∈ {1, 2} do HelpAndPromote(D)
33 C.FAD()
34 return D.isDone()
```

Algorithm 2. Logarithmic-space linear-time abortable Lock algorithm for the oblivious adversary

counts the current contention on the object. The main body of Lock() consists of a while-loop, in lines 25–32. In each iteration of the loop, if D is not done yet and p has not seen an abort signal, then p writes D to a random position of array R sampled from a geometric distribution, in lines 30

and 31. Then, p invokes HelpAndPromote(D) twice, in line 32. Otherwise, if p does see an abort signal, then it executes one final iteration of the while-loop, where it tries to CAS value susp to D.stat in line 27. It then invokes HelpAndPromote(D), which ensures that D is executed if the CAS failed and D.stat = exec.

In call HelpAndPromote(D), the invoking process p first reads the descriptor w stored in L in line 11. Unlike in Algorithm 1, where p just executes w's thunk, p now first attempts to CAS value exec to w.stat, and executes the thunk only if w.stat = exec, in lines 12–14. This prevents the thunk from being executed if w.stat = susp, due to an abort signal. Next, in lines 15–20, p chooses a new descriptor w', which it then attempts to CAS into L in line 21 — if successful, then we say that w' is *promoted*. Process p reads the descriptor d stored at $R[\log k]$, where k is the current value of C (serving as an estimate of point contention), and lets $w' = d$ if d.stat = wait, in lines 16–18. If d.stat $\neq$ wait then p tries to promote its own descriptor D if D.stat = wait, in line 19. Otherwise, $w' = w$ (line 20), and effectively no descriptor is promoted. Thus, if p does promote a new descriptor $w' \neq w$, then w'.stat = wait when p read it.

The algorithm guarantees that call Lock(D) responds after $O(\kappa)$ expected while-loop iterations, where κ is the maximum contention during the call. Next we give a rough proof overview for this result. Suppose that during call Lock(D), the calling process p writes descriptor D to random position $R[g]$ at point τ (in line 31); note that the oblivious adversary does not know the position g. Consider the first HelpAndPromote() call, by any process q, to execute a successful CAS in line 21, such that in this call q reads R after point τ, in line 17, say at point $\tau' > \tau$. This HelpAndPromote() call promotes the descriptor d that q reads from $R[\log k]$ at τ', where k is the contention q observed by reading C in line 16, if d.stat = wait. Therefore, if $g = \log k$ *and* no process writes to $R[\log k]$ in the interval (τ, τ') (and p does not set D.stat to susp), then $d = D$, which means that D will be promoted and subsequently executed. The probability that $g = \log k$ is $\Theta(1/2^{\log k}) = \Theta(1/k)$. Thus, to show an expected number of $O(k)$ loop iterations before Lock() responds, it suffices to argue that with constant probability no process writes to $R[\log k]$ in the interval (τ, τ').

We show this, by arguing that the total number of writes to R in that interval is in $O(k)$. To show this bound on the number of writes, we use the two HelpAndPromote() calls in line 32, and the HelpAndPromote() call in line 24. In particular, the call in line 24 ensures that every process which invokes Lock() after τ, must first execute a HelpAndPromote() call before having a chance of writing to R. Finally, the HelpAndPromote() calls at the beginning of Lock(D), in line 22, strengthen this argument and allow us to use the contention at the beginning of the loop iteration to bound the maximum value of C during the iteration.

We analyze this algorithm's expected step complexity and prove its correctness in Section 4.2. In particular, Theorem 4.18 states that the expected number of steps of a Lock() call is in $O(\mathbf{E}[\kappa] \cdot T)$, where $\mathbf{E}[\kappa]$ is the expected maximum point contention during the call, and T is the maximum step complexity of a thunk.

3.3 Optimal-Time Lock for Adaptive Adversary

The pseudocode of this final algorithm is given in Algorithm 3. We first motivate it by explaining why Algorithm 2 does not work against the weak adaptive adversary. Recall that in Algorithm 2, descriptors are written to array R at an index randomly chosen in line 30, and the index of R used to promote a descriptor is deterministically chosen in line 16 based on the observed contention. The weak adaptive adversary, however, could learn the indices of writes to R, which we refer to as *levels* of R, after writes take place. It could then adaptively change the point contention to correspond to a level different from the level that a victim process randomly chooses to write to, by invoking processes or sending abort signals. The adversary could then repeatedly have processes call Lock(), maintaining this point contention, until the victim's descriptor is overwritten. At that point, it

Shared Data:

- FAID C
- Registers $R[j]$ for $j \in \{0, \ldots, \lfloor \log n \rfloor\}$
- CAS object L with two fields: L.desc and L.level

Initialization:

- $C \leftarrow 0$
- $R[j]$, for $0 \le j \le \log n$, and L.desc reference a dummy descriptor with isDone() = True
- L.level $\leftarrow 0$

HelpAndPromote(descriptor D)

35 $(w, \ell) \leftarrow L$.Read()
36 w.execute()
37 $s \leftarrow D$.isDone()
38 $d \leftarrow R[\ell]$.Read()
39 **if** d.isDone() = False **then** $w' \leftarrow d$
40 **else if** s = False **then** $w' \leftarrow D$
41 **else return**
42 $\lambda \leftarrow \lfloor \log C$.Read()$\rfloor$
43 **with probability** $1/2$ **do**
44 $\quad \ell' \leftarrow$ uniformly random sample from $\{\lambda - 1, \lambda, \lambda + 1\} \cap \{0, \ldots, \lfloor \log n \rfloor\}$
45 **else**
46 $\quad \ell' \leftarrow$ random sample from distribution $\Pr[\ell' = g] = \begin{cases} 2^{-g-1}, & 0 \le g < \lfloor \log n \rfloor \\ 2^{-g}, & g = \lfloor \log n \rfloor \end{cases}$
47 L.CAS($(w, \ell), (w', \ell')$)

Lock(descriptor D)

// each Lock() call takes as argument a distinct descriptor pointing to a distinct thunk
48 C.FAI()
49 **repeat**
50 $\quad \mu \leftarrow \lfloor \log C$.Read()$\rfloor$
51 $\quad$ **with probability** $2^{-\mu-1}$ **do** $R[\mu]$.Write(D)
52 $\quad$ HelpAndPromote(D)
53 **until** D.isDone() = True
54 C.FAD()

Algorithm 3. Logarithmic-space linear-time Lock algorithm for the weak adaptive adversary

allows the victim to take more steps until it writes again. This allows the adversary to starve the victim process while allowing it to take infinite steps.

To avoid this problem, Algorithm 3 makes the choice of level to write to deterministic, and the choice of level to promote random, swapping the random choice of level relative to Algorithm 2. Similarly to Algorithm 1, each write to R occurs with a probability that is a function of the observed contention, although here it is strictly inversely proportional to the observed contention (within constant factors). The new random choice of level is now written into shared variable L as part of an extra field called level, which locks in the next level at the same step that the weak adaptive

adversary learns about it (in line 47). Moreover, the more complicated probability distribution used to choose the new level (lines 42–46) prevents the adversary from starving a victim process for too long—we elaborate in more detail later.

Algorithm 3 has a similar structure as Algorithm 2. Since it is not abortable, it does not need the extra stat field in the descriptors. However, shared variable L has an extra field, level, which stores an index of array R and thus takes values in $\{0, \ldots, \lfloor \log n \rfloor\}$. In the repeat-loop of `Lock()`, in lines 49–53, the invoking process p chooses the level to write to R as the logarithm of the contention observed in C in line 50, and writes with geometric probability, in line 51, similarly to Algorithm 1. In a `HelpAndPromote()` call, helping is done as before, in lines 35 and 36. The descriptor to be promoted is chosen by using the level ℓ stored in L. The descriptor read from $R[\ell]$ in line 38 is promoted, unless its thunk has been already executed, in which case the descriptor of the invoking process is considered next, similarly to Algorithm 2. Finally, the new level to be stored to L, along with the promoted descriptor, is chosen randomly in lines 42–46: it is either the logarithm of the observed contention (± 1), or a sample from a geometric distribution.

The key insight to making this algorithm work is the specific way in which processes choose which level of R to promote with each `HelpAndPromote()` call. When promoting a descriptor to L via a CAS in line 47, processes simultaneously store a level in L, which is then used to decide the level to read from R in line 38 for the subsequent promotion. Since this CAS takes place immediately after the random decision for the next level, it effectively locks in the level for the next promotion. This prevents the weak adaptive adversary from modifying the carefully chosen probability distribution in lines 43–46, explained below. Since the level takes $\Theta(\log \log n)$ bits, and since pointers to heap-allocated objects use only 45 bits [16], the combination of descriptor pointer and level easily fits in a standard 64-byte word.

To explain the probability distribution in lines 43–46, we first highlight that this lock is not abortable, and hence the adversary is limited in the degree to which it can reduce point contention. Specifically, since processes must have their descriptors be promoted before they can exit their `Lock()` calls, the adversary must promote sufficiently many descriptors before it can quickly reduce point contention. Thus, even though the adversary can quickly increase contention at any time, it is limited in its ability to decrease contention.

The random choice of level in line 44 ensures that the next level chosen for promotion is (half the time) close to the level corresponding to the currently observed contention. The random variation of ± 1 in the choice of level accounts for the limited ability of the adversary to modify point contention. The random choice of level in line 46 ensures that a process that writes to R in line 51 gets chosen in a number of promotions proportional to its observed contention, if its descriptor is not overwritten.

Suppose that a process p writes to $R[\alpha]$, after it observes contention of $\Theta(2^\alpha)$. If the adversary does not significantly change contention after p's write, then only $O(2^\alpha)$ processes can overwrite p's descriptor in $R[\alpha]$ before the next promotion. Since they each write with probability $O(2^{-\alpha})$, there is a constant probability that p's descriptor is not overwritten, and a constant probability that α is chosen as the next level in line 44. If the adversary does significantly change contention, then no process attempts to overwrite $R[\alpha]$ because they each read a different value than α in line 50. Thus, p's descriptor is not overwritten for as long as the contention does not return to $\Theta(2^\alpha)$, and so there is a constant probability that level α is chosen in line 46 within 2^α further promotions.

Thus, the probability distribution for the next level effectively deals with the adversary's ability to increase or decrease point contention, at the cost of requiring all `Lock()` calls to run until their descriptors are done.

We analyze this algorithm's step complexity and prove its correctness in Section 4.3. In particular, Theorem 4.37 states that the expected number of steps of a `Lock()` call is in $O(\mathbf{E}[\kappa] \cdot T)$, where κ is the maximum point contention during the call, and T is the maximum step complexity of a thunk.

4 Analyses

4.1 Analysis of Algorithm 1: TryLock for Adaptive Adversary

4.1.1 Correctness.

Observation 4.1. *Let ω be a descriptor. Throughout the execution interval of $\omega.T$, the value of L is ω.*

Proof. Observe that before a process can call ω.`execute()` (in line 2), it reads ω from L in line 1. Hence, $L = \omega$ at some point before the execution interval of $\omega.T$ begins. Moreover, the value of L only changes from ω to a different value if a process executes a successful `CAS()` in line 4, where the first argument is ω. When this happens, the process has already completed an ω.`execute()` call in line 2. □

Observe that mutual exclusion (S1) follows from Observation 4.1. The remaining statements in Section 4.1.1 establish (S2).

We say a `HelpAndPromote()` call by p *promotes* descriptor ω if during this method call p successfully executes a non-trivial `CAS()` in line 4, changing the value of L from some descriptor $\omega' \neq \omega$ to ω.

Observation 4.2. *Suppose at point t_1 descriptor ω gets promoted, and at point $t_2 > t_1$ process p reads L in line 8 of a* `TryLock`(ω) *call. Then ω is done by the time p finishes the execution of line 8.*

Proof. It suffices to show that an ω.`execute()` call completes before p finishes the execution of line 8. If $L = \omega$ at point t_2, when p reads L in line 8, then p itself completes an ω.`execute()` call in the same line. Hence, assume $L \neq \omega$ at point t_2. Since ω gets promoted at $t_1 < t_2$, $L = \omega$ at some point before t_2, and thus at some point $t^* < t_2$ the value of L changes from ω to $\omega' \neq \omega$. Hence, at t^* some process q performs a successful `CAS`(ω, ω') in line 4. Then prior to that, in line 2, process q completes an `execute`(ω) call. □

Lemma 4.3. *If a descriptor ω is promoted or initialized at point t, then*

(a) ω is not written to R after point t,
(b) ω is not promoted after point t, and,
(c) if ω is not the dummy descriptor, then a `TryLock`(ω) *call is active at point t, such that it has executed line 6 and not yet executed line 8.*

Proof. Let t_i for $i \geq 0$ be the point of the i-th promotion in an execution, and let $t_0 = 0$ be the point when L is initialized to the dummy descriptor. Let ω_i be the descriptor promoted at point t_i, with the dummy descriptor referred to as ω_0 for convenience.

For all $i \geq 1$, let q_i be the process that promotes ω_i at point t_i, via a successful CAS from $\omega_{i-1} \neq \omega_i$ to ω_i in line 4. Before point t_i, q_i reads ω_i from R in line 3 at point r_i as part of the same `HelpAndPromote()` call. By the requirements on `TryLock()`, only one process can call `TryLock`(ω_i), and hence only one process can write ω_i into R in line 6; call this process p_i. Let s_i be the last point before r_i when p_i writes ω_i to R: hence, $R = \omega_i$ throughout $(s_i, r_i]$. After point s_i, p_i executes `HelpAndPromote()` twice in line 7, before starting to execute line 8 at point ℓ_i.

We prove the lemma by induction on successive promotions in a given execution. Let $i \geq 0$, and suppose as the inductive hypothesis that the lemma holds for all ω_j with $j < i$. Specifically, if $i \geq 1$, then we suppose that (a) and (b) hold for $\{\omega_0, \ldots, \omega_{i-1}\}$, and if $i \geq 2$, we further suppose that (c) holds for $\{\omega_1, \ldots, \omega_{i-1}\}$. We are thus not making any assumptions for the base case $i = 0$. We will show that (c), (a), and (b) hold, in that order.

(c): Since (c) does not apply to the dummy descriptor, we only need to show it for $i \geq 1$. Applying (b) of the inductive hypothesis for each $j < i - 1$, we have that ω_j is not promoted after t_j, and

therefore $L \neq \omega_{i-1}$ before t_{i-1}. (This result is trivially true for $i = 1$, because t_{1-1} is point 0, when the execution begins.) Since q_i reads ω_{i-1} in line 1 of its `HelpAndPromote()` call that promotes ω_i, it follows that it invokes `HelpAndPromote()` after point t_{i-1}, and in particular, $r_i > t_{i-1}$. Since $s_i < r_i < t_i$, to prove (c), it suffices to show that t_i occurs before p_i's execution of line 8 at point ℓ_i.

Suppose that one of p_i's `CAS()` operations in line 4 succeeds during its two `HelpAndPromote()` calls. (Recall that since this can be a trivial `CAS()` that does not change L's value, this successful `CAS()` is not necessarily a promotion.) If p_i's successful `CAS()` sets the value of L to ω_i, then from applying (b) to all $j < i$, we have that ω_j is not promoted after t_j, and therefore $L \neq \omega_i$ before point t_i. Therefore, p_i's successful `CAS()` of ω_i is no earlier than t_i. If p_i's successful `CAS()` sets the value of L to some other descriptor $\omega' \neq \omega_i$, then it reads ω' from R after $r_i > t_{i-1}$, since $R = \omega_i$ throughout $(s_i, r_i]$. In particular, some process writes ω' to R after point r_i. Applying (a) for each $j < i$, ω_j is not written to R after point $t_j \leq t_{i-1} < r_i$, and so $\omega' \neq \omega_j$. Since p_i's successful `CAS()` sets the value of L to a descriptor not in $\{\omega_0, \ldots, \omega_i\}$, the `CAS()` takes place no earlier than $t_{i+1} > t_i$. In both cases, we have that $t_i < \ell_i$.

Otherwise, p_i has two unsuccessful `CAS()` operations in line 4, one for each `HelpAndPromote()` call. It follows that a promotion $t_{j_1} > s_i$ takes place during p_i's first `HelpAndPromote()` call, and another promotion $t_{j_2} \in (t_{j_1}, \ell_i)$ takes place during p_i's second `HelpAndPromote()` call.

If $t_{j_2} \geq t_i$, then $t_i \leq t_{j_2} < \ell_i$.

The remaining case is that $t_{j_2} < t_i$, but we will show that this is not possible. By applying (b) to all ω_j with $j < j_2 - 1$, we have that $\omega_{j_2-1} \notin \{\omega_0, \ldots, \omega_{j_2-2}\}$ (recall that $j_2 - 1 \geq j_1 > 0$). Applying the earlier arguments for q_i to q_{j_2}, we establish that $r_{j_2} > t_{j_2-1}$, and therefore that $r_{j_2} > t_{j_2-1} \geq t_{j_1} > s_i$. By (b) applied to ω_{j_2}, $\omega_{j_2} \neq \omega_i$, and since $R = \omega_i$ throughout $(s_i, r_i]$, it additionally follows that $r_{j_2} > r_i$, and therefore $t_{j_2} > r_{j_2} > r_i > t_{i-1}$. But this implies that $t_{j_2} \geq t_i$, which is a contradiction, and completes the proof of (c) for $i \geq 1$ from the inductive hypothesis.

(a): For the dummy descriptor ω_0 initialized at the beginning of the execution, (a) holds because processes do not call `TryLock()` with the dummy descriptor, and therefore do not write it to R in line 6. For $i \geq 1$, using (c) for i, p_i has an active `TryLock(`ω_i`)` call in between lines 6 and 8 when ω_i is promoted at point t_i. From Observation 4.2, ω_i is done by the time p_i responds from its `TryLock(`ω_i`)` call, and so p_i returns `True` from its call without writing ω_i again to R after point $s_i < t_i$. By the requirements of calling `TryLock()`, p_i does not call `TryLock(`ω_i`)` again after its call that overlaps t_i, and in particular does not write ω_i to R after t_i. Since only p_i can write ω_i to R, this completes the proof of (a) from the inductive hypothesis.

(b): Up to this point, (a) holds for each $j \leq i$, and by the inductive hypothesis, (b) holds for each $j < i$. To show that (b) holds for i, we will show that $\omega_i \neq \omega_j$ for each $j > i$, by a nested induction over each $j > i$.

The base case of $j = i + 1$ follows immediately from the definition of promotion, which specifies that the value of L *changes* to a distinct descriptor at a promotion, in this case specifically changing from ω_i to $\omega_{i+1} \neq \omega_i$. Also note that at point $r_{i+1} < t_{i+1}$, process q_{i+1} reads $\omega_{i+1} \neq \omega_i$ from R, and therefore, from (a), $R \neq \omega_i$ at all points after r_{i+1}.

For the inductive case of $j > i + 1$, we assume that $\omega_{j-1} \neq \omega_i$, and show that $\omega_j \neq \omega_i$. Since process q_j changes L from ω_{j-1} to ω_j at point t_j, it reads ω_{j-1} from L in line 1 before point $r_j < t_j$, when it reads ω_j from R in line 3. If $i \geq 1$, then from the outer inductive hypothesis, (b) holds for each $k < i$, so $\omega_{j-1} \neq \omega_k$ for each $k \in \{0, \ldots, i-1\}$. From the nested inductive hypothesis, we also have that $\omega_{j-1} \neq \omega_i$. Therefore, $L \neq \omega_{j-1}$ before point t_{i+1}, so process q_j reads ω_{j-1} from L after point t_{i+1}, and so $r_j > t_{i+1} > r_{i+1}$. Since $R \neq \omega_i$ after point r_{i+1}, it follows that process q_j reads a descriptor different from ω_i at point $r_j > r_{i+1}$, which establishes that $\omega_j \neq \omega_i$.

This completes the proof of (b) for i, and the proof of the lemma for all $i \geq 0$. □

Theorem 4.4. *The algorithm satisfies (S1) and (S2).*

Proof. Mutual exclusion (S1) follows immediately from Observation 4.1 and the fact that L can always store only one descriptor.

For (S2), consider a non-dummy descriptor ω whose thunk gets executed, and let I be the execution interval of that thunk. Since ω is not the dummy descriptor, some process q promotes ω in line 4 at some point t_{prom} which is before I starts. From Lemma 4.3(c), a process p has a pending `TryLock`(ω) call at point t_{prom}, such that it has not yet executed line 8. By Observation 4.2, ω is done before p can finish its `TryLock()` call, and hence I is entirely during p's `TryLock`(ω) call.

Finally, it follows from line 10 that a `TryLock()` call returns True if and only if its descriptor's task is done. This completes the proof of (S2). □

4.1.2 Success Probability.

Lemma 4.5. *Let $t < t'$ be two points in time, such that at point t process p writes descriptor ω to R, during $(t, t']$ some process q performs two complete* `HelpAndPromote()` *calls, and ω does not get promoted at or before t'. Then at some point $t^* \in (t, t')$, a descriptor $\omega^* \neq \omega$ is written to R.*

Intuitively, if q executed only one `HelpAndPromote()` call in line 7, then its `CAS()` in line 4 could fail due to another descriptor ω' getting promoted in between lines 1 and 4. But ω' could have been read from R before p wrote ω to R in line 6, so this would not be sufficient to prove Lemma 4.5. As such, the second `HelpAndPromote()` ensures that there is a successful `CAS()` on L which reads a descriptor from R after p's write in line 6 (either by q, or by another process which causes q's second CAS attempt to fail).

Proof. Suppose that one of q's `CAS()` operations on L in line 4 is successful at point t_C, and sets the value of L to some descriptor ω' that q earlier reads in line 3 at point $t_R \in (t, t_C)$. Since $t_C \leq t'$, it follows that ω' is promoted no later than t' (it could be promoted before t_C, if q's `CAS()` was from ω' to itself). Since ω' does not get promoted at or before t', $\omega' \neq \omega$. Then, since $t_R > t$, it follows that $\omega' \neq \omega$ was written to R at some point after t and before t'.

Otherwise, assume that q's `CAS()` operations on L in line 4 fail in both `HelpAndPromote()` calls. Since the old value used for the `CAS()` in line 4 is the descriptor read from L in line 1, the value of L changes at least once during each of q's two `HelpAndPromote()` calls. Let t_1 and t_2 be the first two points after t at which the value of L changes; call the two corresponding promoted descriptors ω_1 and ω_2. Since ω is not promoted at or before $t' > t_2$, $\omega \notin \{\omega_1, \omega_2\}$. From Lemma 4.3(b), $L \neq \omega_1$ before t_1. Hence, the `HelpAndPromote()` call h_2 that promotes ω_2 at t_2 is invoked after t_1, in order to read ω_1 from L in line 1 and have a successful `CAS`(ω_1, ω_2) in line 4. Thus, h_2 reads ω_2 from R in line 3 after point $t_1 > t$. Therefore, $\omega_2 \neq \omega$ is written to R after point t. □

Observation 4.6. *Whenever k processes have a pending* `TryLock()` *call that has not yet executed line 9, $C \geq k$.*

Proof. Recall that a `TryLock()` call is pending when a process has performed its first but not yet its last shared memory operation of the call. Since the first shared memory operation is C.`FAI()` and each process has not yet executed C.`FAD()` in line 9, and no other line affects the value of C, each of the k processes has performed more C.`FAI()` than C.`FAD()` operations. This implies the claim. □

Corollary 4.7. *Suppose at some point processes $p_1, p_2, \ldots, p_k$ have pending* `TryLock()` *calls that have not yet executed line 9, in the order in which they invoke their* `TryLock()` *calls. Then p_i observes contention at least i.*

Proof. This follows from Observation 4.6, because when process p_i invokes its `TryLock()` call, processes $p_1, \ldots, p_{i-1}$ have pending `TryLock()` calls which have not yet executed line 9. Hence, p_i's C.`FAI()` in line 5 returns a value of at least $i - 1$. □

Lemma 4.8. *Suppose at point t process p writes to R in line 6 during a* `TryLock(`ω`)` *call. Then with probability at least 1/2, the* `TryLock()` *call is successful when it terminates.*

Proof. Consider the execution prefix that ends at point t, and observe from Lemma 4.3(a) that ω is not promoted before t. At this point, we fix an infinite number of biased coin flips $c_1, c_2, \ldots \in \{0, 1\}$ where each c_i is chosen independently at random such that $\text{Prob}(c_i = 1) = \varepsilon_i$, where $\varepsilon_i = 1/(7i \log^2(i+1))$.

Now consider an arbitrary execution that has the same prefix up to and including point t, and which ends when p's `TryLock(`ω`)` call terminates. Let $p_1, p_2, \ldots$ be the processes that execute line 6 after point t, in the order in which they invoke their `TryLock()` calls. Let t^* be the first point after t at which either descriptor ω gets promoted or some process writes to R. By Lemma 4.5, one of the two must happen before p executes line 8.

We will use the following observation:

> Suppose that p_i executes line 6 at point $t_i > t$, such that ω is not promoted before t_i, and that no process writes to R in (t, t_i). Then, the probability that p_i writes to R at t_i, given the execution up to just before t_i, is at most ε_i. (1)

To see that this is true, consider a process p_i that executes line 6 during $[t, t^*]$. Then before p_i executes line 9, p_i completes two `HelpAndPromote()` calls, and thus by Lemma 4.5, either ω gets promoted or some process writes to R. It follows from the definition of t^* that p_i does not execute line 9 before t^*. Since this holds for each process p_j that executes line 6 during $[t, t^*]$, each such process has a pending `HelpAndPromote()` call at point t^* which has not yet executed line 9. Thus, by Corollary 4.7, p_i observes contention at least i, and so the probability that it writes to R in line 6 at point t_i is at most $1/(7i \log^2(i+1)) = \text{Prob}(c_i = 1)$. (Recall from Section 2.3 that the weak adaptive adversary schedules processes based on knowledge of the full execution up to each point, but cannot intervene between random decisions and the immediate next shared memory operation.)

We now assume that if process p_i executes line 6 during $[t, t^*]$, then it uses the predetermined value of c_i as its coin flip, i.e., it writes if and only if $c_i = 1$. We can make this assumption without loss of generality, because by (1) the probability that p_i actually writes in that line can only be lower. Note also that, since the probability in (1) is conditioned on the execution up to just before p_i's execution of line 6, this upper bound probability is independent of all prior random decisions.

Let A be the event that all coin flips are 0, i.e., $0 = c_1 = c_2 = \ldots$. If A occurs, then no process writes to R in $[t, t^*]$. Hence, by definition of t^*, ω gets promoted at t^*. It follows that the probability that ω gets promoted before p executes line 8 is at least

$$\text{Prob}(A) \geq 1 - \text{Prob}(\exists i : c_i = 1) \geq 1 - \sum_{i=1}^{\infty} \frac{1}{7i \log^2(i+1)} \geq 1/2.$$

The lemma then follows from Observation 4.2. □

Theorem 4.9. *Each* `TryLock()` *call in Algorithm 1 succeeds with probability $\Omega\left(\frac{1}{k \log^2(k+1)}\right)$, where k is the contention at the invocation of the call.*

Proof. Let t be the point when process p executes line 6 after observing contention k in line 5. If p writes to R at point t, then by Lemma 4.8, with probability at least 1/2 process p's `TryLock()` call will be successful. Since p writes with probability $\Omega(1/(k \log^2(k+1)))$, the statement follows. □

4.2 Analysis of Algorithm 2: Abortable Lock for Oblivious Adversary

4.2.1 Correctness.

Observation 4.10. *For any descriptor ω, if ω.stat $= s \neq$ wait at point t, then ω.stat $= s$ throughout $[t, \infty)$.*

Proof. This follows immediately from the fact that the only operations which can change a stat field change it from wait to either exec (in line 12) or susp (in line 27). □

Lemma 4.11. *Suppose that there exist points t_1 and $t_2 > t_1$ such that $L = \omega$ for some descriptor ω at point t_1, and there is a successful CAS on L at point t_2. Then, throughout $[t_2, \infty)$, either*

- *ω.stat = susp, or*
- *ω.stat = exec and ω is done.*

Proof. Using Observation 4.10, ω.stat can only change once from wait. Therefore, we only need to prove the lemma for point t_2. We can also assume without loss of generality that t_2 is the point of the *first* successful CAS on L after t_1, and therefore that $L = \omega$ just before t_2.

Let p be the process that executes the successful CAS on L at point t_2. Since $L = \omega$ just before p's successful CAS, it follows that p reads ω from L in line 11. Since process p attempts a CAS on ω.stat from wait to exec in line 12 at some point $t^\dagger < t_2$, by Observation 4.10 it follows that ω.stat $\neq$ wait just after $t^\dagger$. If ω.stat = susp at this point, then the lemma follows immediately, so suppose that ω.stat = exec. Since p then reads ω.stat = exec in line 13 after $t^\dagger$, it executes ω's thunk in line 14 before t_2, and hence ω is done by point t_2. □

Lemma 4.12. *If ω.stat = susp at any point in the execution, then ω.`execute()` is never called.*

Proof. Consider any process p that reads ω from L in line 11. In line 12, p then attempts a CAS from ω.stat from wait to exec.

If ω.stat $\neq$ susp immediately before the CAS, then ω.stat = exec after the CAS, and from Observation 4.10 it remains exec for the rest of the execution. But Observation 4.10 also implies that once ω.stat = susp, it remains susp for the rest of the execution, which is a contradiction.

It follows that ω.stat = susp immediately before p's CAS, the CAS fails, and ω.stat = susp $\neq$ exec when p reads it in line 13. Thus, p does not execute line 14 with $w = \omega$. Since this is true for any p, and since thunks can only be executed from line 14, the lemma follows. □

Recall that a thunk τ's *execution interval* start at the first invocation of any ω.`execute()` call and ends at the first response of any ω.`execute()` call.

Lemma 4.13. *Given a descriptor ω, if ω.`execute()` is called, then $L = \omega$ throughout $\omega.T$'s execution interval.*

Proof. Recall that ω.`execute()` can only be called in line 14, after reading ω from L in line 11. Let t_1 be the first point at which L changes to ω. Then, every invocation of ω.`execute()` happens after point t_1. Let t_2 be the first point after t_1 at which L changes to a value different from ω (if this point does not exist, then the lemma follows immediately). Since ω is executed, from Lemmas 4.11 and 4.12 it follows that ω.stat = exec and ω is done at point t_2. Therefore, some process completes a call of ω.`execute()` in line 14 before point t_2.

Since the first invocation of ω.`execute()` is after t_1 and the first response of ω.`execute()` is before t_2, and since $L = \omega$ throughout the interval $[t_1, t_2)$ the lemma follows. □

We show (S1), mutual exclusion of thunk execution intervals, as a result of Lemma 4.13.

Lemma 4.14. *For any two distinct thunks $\tau_1 \neq \tau_2$ that are executed, their execution intervals are disjoint.*

Proof. Recall that each thunk is uniquely associated with a descriptor, so there are distinct descriptors ω_1 and ω_2 which store thunks τ_1 and τ_2, respectively. From Lemma 4.13, $L = \omega_i$ throughout $\omega_i.T$'s execution interval, for $i \in \{1, 2\}$. The lemma follows from the fact that $\omega_1 \neq \omega_2$. □

Finally, we show (S2), that the response of `Lock(`ω`)` calls correctly indicate whether the thunk is executed, and that thunk executions only occur within a corresponding `Lock()` call.

Lemma 4.15. *If a* `Lock(`ω`)` *call responds at point t, then either*

- *The call returns* True*, and ω is done at point t, or*
- *The call returns* False*,* abort_sig = True *at some point before t, and ω*`.execute()` *is never called.*

Proof. Suppose that a `Lock(`ω`)` call by process p call returns True at point t. Then ω is done at point t, which can only happen after some invocation of ω`.execute()` responds, proving the first part of the lemma.

Otherwise, suppose that the `Lock(`ω`)` call by process p call returns False at point t. Then there is some point $t' < t$ at which p breaks out of its while-loop in line 25 is false. Since ω is not done at point t, it is not done at all points before t, and hence p exits its loop because D.stat = susp at point t'. The second part of the lemma then follows from Lemma 4.12. □

4.2.2 Expected Step Complexity.

Adversary For the analysis of Algorithm 2 only, we use a slightly stronger adversary than the standard oblivious adversary. Specifically, we assume that the adversary behaves like the weak adaptive adversary defined in Section 2.3, except that it never (directly) learns the value of any local variable, and also never learns the contents of the array R.

The following is the main lemma of our step complexity analysis.

Lemma 4.16 (Probability of Promotion). *Suppose that process p writes descriptor ω to an entry of R at point τ as part of a* `Lock(`ω`)` *call. Let t_{loop} be the point when p completes the while-loop iteration containing point τ, and let κ be the maximum point contention in p's* `Lock(`ω`)` *call from invocation until point τ. Given the execution up to point τ except for the location of p's write at point τ, the probability that $L = \omega$ at or before the point t_{loop} is at least $1/(2e\kappa)$.*

We prove Lemma 4.16 in Section 4.2.3.

Lemma 4.17. *Suppose that process p invokes a* `Lock(`ω`)` *call for some descriptor ω. If there is a point t during the call at which either $L = \omega$ or* abort_sig = True *for the first time, then p takes a constant number of non-thunk steps after point t until its* `Lock(`ω`)` *call responds.*

Proof. Observe that the `HelpAndPromote()` method executes at most one thunk, and takes a constant number of other shared memory steps. Moreover, the `Lock()` method's while loop in lines 25–32 consists of at most a constant number of `HelpAndPromote()` calls and other shared memory operations, and takes only two shared memory operations after the loop completes. Suppose that p calls `HelpAndPromote()` at least once after point t, otherwise the lemma follows immediately.

If abort_sig = True at point t, then after a constant number of p's non-thunk steps, p sees abort_sig = True in line 26. In the next step, p attempts a CAS on ω.stat from wait to susp in line 27. If ω.stat = susp immediately after the CAS, then by Observation 4.10 p sees ω.stat = susp in its

next execution of line 25 and hence exits the while-loop. If ω.stat $\neq$ susp immediately after the CAS, then the only remaining possibility is that ω.stat = exec immediately after (and before) the CAS. Recall that only line 12 can change ω.stat to exec, and it can only be called after a point when $L = \omega$. Thus, there is a point t' at which $L = \omega$, and which is at most a constant number of p's non-thunk steps after point t, for which the second case (proven below) applies.

It remains to show the lemma for the case where $L = \omega$ at point t. Let $t^\star$ by the final step of the first `HelpAndPromote()` call that p starts after point t. Either p's CAS in line 21 succeeds at point $t^\star$, or L changes at some point in $(t, t^\star)$. Using Lemma 4.11, by point $t^\star$, either ω.stat = susp or ω is done, and so p exits the while-loop within a constant number of non-thunk steps. □

Theorem 4.18. *In Algorithm 2, the expected number of steps of a* `Lock()` *call is in* $O(\mathbf{E}[\kappa] \cdot T)$*, where* κ *is the maximum point contention during the call, and* T *is the maximum step complexity of a thunk.*

Proof. Suppose that process p invokes a `Lock`(ω) call. From Lemma 4.17, it suffices to bound p's expected step complexity until the first point when either $L = \omega$ or abort_sig = True.

Suppose that the adversary either schedules p for sufficiently many steps to either respond from its `Lock`(ω) call, or schedules p for infinitely many steps. For $i \geq 1$, let κ_i be the maximum point contention of the `Lock`(ω) call until the point when p executes its i-th write to R, or until p responds if it writes to R fewer than i times. (In particular, if p sees the abort signal in iteration i, then by the proof of Lemma 4.17, p breaks out of the while loop after iteration i, and κ_i is the maximum point contention of the entire `Lock`(ω) call.) Since p contributes to point contention, it follows that $\kappa_i \geq 1$ for all i. Let T be the total number of loop iterations of the `Lock`(ω) call, before it responds. Observe that κ_i and T are random variables, given the execution up to the point that p invokes `Lock`(ω). For $i \geq 1$, define the random variable Y_i such that

$$Y_i = \begin{cases} 1, & \text{if } i < T \\ 1 - 2e\kappa_i, & \text{if } i = T \\ 0, & \text{if } i > T. \end{cases}$$

For a given i, it is possible to infer whether $i \geq T$ from the partial sequence $Y_1, \ldots, Y_i$; thus, T is a stopping time. We also define $X_i = \sum_{j \leq i} Y_j$ for $i \geq 0$.

Using Lemma 4.16, we show that the sequence $X_0, X_1, \ldots$ is a supermartingale. For $i \geq 1$,

$$\mathbf{E}[Y_i \mid i \leq T, \kappa_i = k, Y_1, \ldots, Y_{i-1}] \leq 1 \cdot \left(1 - \frac{1}{2ek}\right) + (1 - 2ek) \cdot \frac{1}{2ek} = 0, \text{ and}$$
$$\mathbf{E}[Y_i \mid i > T, Y_1, \ldots, Y_{i-1}] = 0, \text{ thus}$$
$$\mathbf{E}[Y_i \mid Y_1, \ldots Y_{i-1}] \leq 0.$$

Recall that the implementation makes use of a specific bound n on the number of processes, in particular to determine the size of register array R. By Lemma 4.16, we have that $\mathbf{E}[T] \leq 2en$. For all i, $|Y_i| \leq 2en - 1$. Hence, we can use the optional stopping theorem to obtain that

$$\mathbf{E}[X_T] \leq X_0 = 0.$$

We also have that

$$X_T = T - 2e\kappa_T \geq T - 2e\kappa,$$

where κ is the maximum contention during the `Lock()` call. It follows that

$$\mathbf{E}[T] - 2e\,\mathbf{E}[\kappa] \leq \mathbf{E}[X_T] \leq 0, \text{ which gives}$$
$$\mathbf{E}[T] \leq 2e\,\mathbf{E}[\kappa].$$

□

4.2.3 Proof of Lemma 4.16. We begin by proving a key feature of the `HelpAndPromote()` method.

Lemma 4.19. *Suppose that a process p executes a* `HelpAndPromote(`ω`)` *method call that begins at point t_1 and ends at point t_2. If the value of L does not change in the interval $[t_1, t_2]$, then*

- *p's CAS in line 21 succeeds (with identical old and new values, not changing L), and*
- *throughout $[t_2, \infty)$, either ω is done or ω.*stat = susp.

Proof. Let ω_L be the descriptor that p reads from L in line 11 at point t_1. Since we assume that L does not change throughout $[t_1, t_2]$, and since p's CAS in line 21 at point t_2 uses ω_L as the old value, it follows that p's CAS is successful. Since L does not change at point t_2 in particular, it also follows that t_2 is the point of a successful CAS from ω_L to itself.

Consider first the case where $\omega_L = \omega$. Since $L = \omega$ at point t_1 and p's CAS on L is successful at point t_2, the lemma follows from Lemma 4.11.

It remains to show the case where $\omega_L \neq \omega$.

After p attempts a CAS on ω_L.stat from wait to exec in line 12, ω_L.stat $\neq$ wait, and from Observation 4.10 it remains unchanged for the rest of the execution. In particular, ω_L.stat $\neq$ wait when p executes lines 15 and 18.

If the if-condition in one of line 19 and 18 returns true, then p sets its local variable w' to a descriptor whose stat field is wait after p's CAS in line 12. In that case, $w' \neq \omega_L$, contradicting that p's CAS in line 21 uses ω_L as both new and old values. Therefore, both if-conditions in lines 19 and 18 return false. In particular, ω.stat $\neq$ wait when p reads it in line 15 at point t_s.

If ω.stat = susp at point t_s, then the lemma follows from Observation 4.10, so suppose that ω.stat = exec at point t_s, and therefore that ω.stat $\neq$ susp throughout the execution. Thus, some process q changes ω.stat from wait to exec via a successful CAS in line 12 at some point $t^\star < t_s < t_2$. Before point $t^\star$, q reads ω from L in line 11. Since $L = \omega_L$ throughout $[t_1, t_2]$, there is a point $t^\dagger < t_1$ at which L changes from ω via a successful CAS in line 21. Using Lemma 4.11 and that ω.stat $\neq$ susp throughout the execution, ω is done throughout $[t_1, \infty) \supset [t_2, \infty)$. □

Lemma 4.20. *If $L = \omega$ at points t_1 and $t_2 > t_1$, then $L = \omega$ throughout the interval $[t_1, t_2]$.*

Proof. Suppose for the purpose of proving a contradiction that there exist points t_1 and t_2 such that $L = \omega$ at t_1 and t_2, but there is some point in the interval (t_1, t_2) at which $L \neq \omega$. Without loss of generality, suppose that t_2 is also the earliest point in the execution in which L changes back to any value that it held at an earlier point. Thus, some process p executes a successful CAS in line 21 from some value $\omega^\dagger \neq \omega$ to ω at point t_2.

Let t' be the point when p reads $\omega^\dagger$ from L in line 11. Since t_2 is the first point when L changes back to a previous value, and since p's CAS at point t_2 succeeds, it follows that $L = \omega^\dagger$ throughout $[t', t_2)$. Since $L = \omega \neq \omega^\dagger$ at point $t_1 < t_2$, it follows that $t_1 < t'$. Moreover, there is a point $t^\star \in (t_1, t')$ at which L changes via a successful CAS from ω to some value $\omega^\star \neq \omega$ (possibly $\omega^\star = \omega^\dagger$). From Lemma 4.11, ω.stat $\neq$ wait throughout the interval $[t^\star, \infty) \supset (t', t_2)$.

Since $\omega^\dagger \neq \omega$, it follows that p's local variable $w' = \omega$ does not equal its local variable $w = \omega^\dagger$ at the point of p's CAS in line 21. In particular, p does not execute line 20, and so either the if-condition in line 18 or that in line 19 succeeds. But in both cases, p sets w' to a descriptor whose stat field is wait at some point after t', which contradicts that p's CAS in line 21 changes L to descriptor ω, whose stat field is not wait throughout (t', t_2). □

Definition 4.21. *Let t_{CAS} be the point of the first successful CAS on L in line 21 by a process that reads R in line 17 after point τ, and call this process q. (We will later show that such a point exists as long as the adversary schedules p for enough steps.) We define the following points from q's* `HelpAndPromote()` *call that finishes at point t_{CAS}:*

- t_L *is the point of* q*'s read of* L *in line 11,*
- t_C *is the point of* q*'s read of* C *in line 16, and*
- t_R *is the point of* q*'s read of* R *in line 17.*

It follows from Definition 4.21 that $t_L < t_C < t_R < t_{CAS}$, and $\tau < t_R$.

Observation 4.22. *The value of* L *does not change in the interval* $[t_L, t_{CAS})$.

Proof. Since t_{CAS} is the point of a successful CAS, it follows that the value of L is the same at point t_L and just before point t_{CAS}. The observation then follows from Lemma 4.20. □

Definition 4.23. *Let* t_0 *and* t_1 *be the last points prior to* τ *and* t_{CAS}*, respectively, when the value of* L *changes or is initialized.*

Since $t_0 < \tau < t_R < t_{CAS}$, it follows that $t_0 \le t_1$. From Observation 4.22, we also have that $t_1 < t_L$. If $t_0 \ne t_1$, then by definition of t_0, $\tau < t_1$.

Lemma 4.24. *If* $t_0 \ne t_1$*, then the value of* L *does not change in the interval* (t_0, t_1).

Proof. Let r be the process that executes a successful CAS on L in line 21 at point t_1. Let t' be the last point when r executes line 11 before t_1. Since $t_1 < t_L < t_{CAS}$, it follows that r executes line 17 before point τ, and therefore that $t' < \tau$. Since r's CAS is successful at point t_1, it follows from Lemma 4.20 that the value of L does not change in the interval $[t', t_1)$. The lemma follows from the fact that $[t', t_1)$ overlaps with the interval $(t_0, \tau]$, during which the value of L also does not change, by definition of t_0. □

Lemma 4.25. *If a process invokes the* `Lock()` *method after point* t_0*, then it does not increment* C *until after* t_C.

Proof. Suppose that process r invokes a `Lock(`ω_r`)` call after t_0 and increments C at point t_{inc}. Since only r can set ω_r.stat to susp, and it can only do so in line 27 after incrementing C in line 23, it follows that ω_r.stat $\ne$ susp before t_{inc}. Similarly, only r can write ω_r to R, and it can only do so in line 31 after incrementing C in line 23, so R does not contain ω_r before t_{inc}. In particular, no process other than r can change L to ω_r before t_{inc}, since, from lines 17–21, processes only change L to either a descriptor they read from R or their own unique descriptor. Recall that ω_r`.isDone()` can only return True after ω_r.stat becomes exec, which in turn can only happen after the first point when $L = \omega_r$. It follows that ω_r is not done until at least the end of r's first `HelpAndPromote(`ω_r`)` call. Therefore, at the last step of r's first `HelpAndPromote(`ω_r`)` call, ω_r.stat $\ne$ susp, and ω_r is not done. By Lemma 4.19, it follows that L changes at some point $t^\star \in (t_0, t_{inc})$ during r's first `HelpAndPromote(`ω_r`)` call.

If $t_0 = t_1$, then by definition of t_1, L does not change in the interval (t_0, t_{CAS}), and so $t^\star \ge t_{CAS}$. Hence, r increments C after $t_{CAS} > t_C$, proving the lemma. Otherwise, by Lemma 4.24, $t^\star \ge t_1$.

Now consider r's second `HelpAndPromote(`ω_r`)` call of line 22. Either r's CAS in line 21 succeeds, or it fails because some other CAS on L successfully changes L after r reads it in line 11 as part of the call. In both cases, there is a successful CAS on L after t_1 and before t_{inc} by some process ρ. By Lemma 4.20, ρ reads L in line 11 after $t_1 > \tau$, and therefore its successful CAS is later or equal to $t_{CAS} > t_C$, which again implies that r increments C after t_C. □

Lemma 4.26. *The value of* C *at point* t_C *is at most the maximum point contention in* $(t_0, \tau]$.

Proof. We show that the value of C at point t_C is at most the maximum point contention at point $\min\{t_C, \tau\} \in (t_0, \tau]$.

If $t_C < \tau$, then the lemma follows immediately from the fact that each active process contributes one to the point contention and at most one to the value of C.

Otherwise, $\tau < t_C$. In this case, consider the processes that contribute one to the value of C at point t_C. If such a process is invoked before point τ, then it contributes one to the point contention at point τ. Otherwise, it is invoked after point $\tau > t_0$, but by Lemma 4.25, it does not increment C until after t_C, contradicting that it contributes one to the value of C at point t_C. □

Lemma 4.27. *Suppose that process r writes to R at some point $t \geq \tau$. Then r does not complete its while-loop iteration containing point t before point t_{CAS}.*

Proof. Let t' be the point in which r finishes the while-loop iteration that includes its write at point t. During the interval $(t, t']$, process r executes `HelpAndPromote()` twice in line 32. Applying Lemma 4.19 twice, either r has a successful CAS on L at some point in $(t, t']$, or the value of L changes twice in (t, t').

If r has a successful CAS on L at point $t^\star \in (t, t']$, then because r reads R in line 17 after point τ, it follows that $t_{CAS} \leq t^\star \leq t'$.

Otherwise, there exist points c_1 and c_2 such that L changes at c_1 and c_2, and $\tau \leq t < c_1 < c_2 < t'$. The process that changes L at point c_2 executes a successful CAS in line 21, and by Lemma 4.20 it executes line 11 after point c_1, and hence executes line 17 after point τ. Thus, t_{CAS} is earlier or equal to $c_2 < t'$. □

An immediate corollary of Lemma 4.27 is that t_{CAS} and prior points exist as long as the adversary schedules p for enough steps to complete its while-loop iteration containing point τ.

Lemma 4.28. *Suppose that a process r writes to R at point $t \in [\tau, t_{CAS})$, and let t_i be the point of the last point prior to t at which L changes or is initialized. (Since $t < t_{CAS}$, if $\tau < t_1 < t$, then $t_i = t_1$, and otherwise $t_i = t_0$.) Then, r does not*

- *write to R or decrement C in $(t, t_{CAS}]$, or*
- *increment C in $[t_i, t]$.*

Proof. We first prove that r neither writes to R nor decrements C in $(t, t_{CAS}]$. Let t' be the point in which r finishes the while-loop iteration that includes its write at point t. From the code, t' happens before the first point after t at which r can either write to R again or decrement C. (If t' does not exist, then r never writes to R or decrements C after t.) From Lemma 4.27, $t_{CAS} \leq t'$.

It remains to show that r does not increment C in $[t_i, t]$. Observe that r executes `HelpAndPromote()` in line 24 after incrementing C and before point t. From Lemma 4.19, either L changes during this `HelpAndPromote()` call, or r's while-loop condition in line 25 becomes false by the end of the call and remains false for the rest of the execution. The latter case would contradict that r writes to R at point t, and therefore the value of L changes during r's `HelpAndPromote()` call in line 24, at point t_i. □

Applying Lemma 4.28 to p's write at point τ, we obtain that p's invocation of `Lock(`ω`)` takes place before t_0.

Lemma 4.29. *The number of writes to R in (τ, t_R) is less than the value of C at point t_C.*

Proof. Recall that $t_1 < t_C < t_R < t_{CAS}$. From Lemma 4.28, every process that writes to R in the interval $[\tau, t_R)$ does so exactly once, and contributes one to the value of C throughout the interval $[t_1, t_{CAS}] \ni t_C$. The strict inequality follows from excluding p's write at point τ from the interval, and that other process that don't write in the interval can only increase the value of C. □

Lemma 4.30. *Given the execution up to point τ except for the location of p's write at τ, and that the value of C at point t_C is k, the probability that $L = \omega$ at or before point t_{CAS} is at least $1/(2ek)$.*

Proof. Suppose that $L \neq \omega$ at all points before t_{CAS}, as this can only decrease the given probability. It follows from this supposition that L.stat $\neq$ exec before t_{CAS}, since processes only change a stat field to exec after reading the descriptor from L.

Since each process calls Lock() with distinct descriptors, only p can change ω.stat to susp, and only via a successful CAS in line 27 after receiving the abort signal. Since abort_sig does not change from True to False after such a CAS, p never writes ω to R after receiving the abort signal. It follows from Lemma 4.27 that p does not attempt a CAS on ω.stat to susp at any point before t_{CAS}. Thus, using Observation 4.10, ω.stat = wait at all points before t_{CAS}.

Recall that the adversary cannot directly observe the contents of R, and that values from R only affect L via a successful CAS in line 21, and do not directly affect anything else. Since t_{CAS} is the point of the first CAS on L whose read of R occurred after point τ, it follows that all writes to R at and after point τ do not affect L before point t_{CAS}. Therefore, just before point t_{CAS}, the adversary has no information about the indices chosen in line 30 which were then written in line 31 in the interval $[\tau, t_R)$.

If p writes ω to $R[\lfloor \log k \rfloor]$, and if no write to R in the interval (τ, t_R) chooses index $\lfloor \log k \rfloor$ (overwriting ω), then process q reads ω from R at point t_R. In this case, since ω.stat = wait at all points before t_{CAS}, it follows that q's successful CAS at point t_{CAS} changes the value of L to ω. By Lemma 4.29, the number of writes to R in (τ, t_R) is less than k, i.e., at most $k-1$. Therefore, the probability that ω is stored in L at point t_{CAS} is at least

$$\begin{aligned} & 2^{-\lfloor \log k \rfloor - 1} \cdot \left(1 - 2^{-\lfloor \log k \rfloor - 1}\right)^{k-1} \\ \geq\ & 2^{-\log k - 1} \cdot \left(1 - 2^{-\log k}\right)^{k-1} \\ =\ & \frac{1}{2k} \cdot \left(1 - \frac{1}{k}\right)^{k-1} \\ \geq\ & \frac{1}{2ek}, \end{aligned}$$

where we use that p contributes to k, hence $k \geq 1$. □

From Lemma 4.27, $t_{CAS} \leq t_{loop}$. From Lemma 4.28, p's Lock(ω) call is invoked before point t_0. Thus, κ is larger than or equal to the maximum point contention in the interval $(t_0, \tau]$, and from Lemma 4.26, this contention is in turn larger than or equal to the value of C at point t_C. Lemma 4.16 then follows from Lemma 4.30.

4.3 Analysis of Algorithm 3: Lock for Adaptive Adversary

4.3.1 Correctness. Note that this section closely follows Section 4.2.1, but is simpler since this implementation is not abortable.

Lemma 4.31. *Suppose that there exist points t_1 and $t_2 > t_1$ such that L.desc = ω for some descriptor ω at point t_1, and there is a successful CAS on L at point t_2. Then, throughout $[t_2, \infty)$, ω is done.*

Proof. It suffices to prove the lemma for point t_2. We can assume without loss of generality that t_2 is the point of the *first* successful CAS on L after t_1, and in particular that L.desc = ω just before t_2.

Let p be the process that executes the successful CAS on L at point t_2. Since L.desc = ω just before p's successful CAS, it follows that p reads ω from L in line 35. Since p then executes ω's thunk in line 36 before t_2, it follows that ω is done by point t_2. □

As with Lemma 4.13, recall that the *execution interval* of a descriptor ω's thunk starts at the first invocation of any ω.execute() call and ends at the first response of any ω.execute() call.

Lemma 4.32. *Given a descriptor ω, L.desc $= \omega$ throughout ω.T's execution interval.*

Proof. Recall that ω.execute() can only be called in line 36, after reading ω from L in line 35. Let t_1 be the first point at which L.desc changes to ω. Then, every invocation of ω.execute() happens after point t_1. Let t_2 be the first point after t_1 at which a successful CAS changes the value of L (if this point does not exist, then the lemma follows immediately). From Lemma 4.31 it follows that ω is done at point t_2. Therefore, some process completes a call of ω.execute() in line 36 before point t_2.

Since the first invocation of ω.execute() is after t_1 and the first response of ω.execute() is before t_2, and since L.desc $= \omega$ throughout the interval $[t_1, t_2)$ the lemma follows. □

We show (S1), mutual exclusion of thunk execution intervals, as a simple corollary of Lemma 4.32.

Corollary 4.33. *For any two distinct thunks $\tau_1 \neq \tau_2$ that are executed, their execution intervals are disjoint.*

Proof. Recall that each thunk is uniquely associated with a descriptor, so there are distinct descriptors ω_1 and ω_2 which store thunks τ_1 and τ_2, respectively. From Lemma 4.32, L.desc $= \omega_i$ throughout ω_i's execution interval, for $i \in \{1, 2\}$. The corollary follows from the fact that $\omega_1 \neq \omega_2$. □

Finally, the below lemma implies (S2), that thunk executions only occur within a corresponding Lock() call.

Lemma 4.34. *If a* Lock(ω) *call responds at point t, then ω is done at point t.*

Proof. Suppose that a Lock(ω) call by process p call responds at point t. Then there is a point $t' < t$ at which p breaks out of its repeat-until loop in line 53 by seeing that ω.isDone() returns True. □

4.3.2 Expected Step Complexity. We begin by stating the main lemma used to prove step complexity.

Lemma 4.35 (Probability of Eventual Promotion). *Suppose that process p writes descriptor ω_p to $R[\alpha]$ at point τ as part of a* Lock(ω_p) *call. For $i \geq 1$, let t_i denote the point of the i-th successful L.CAS() operation after τ. Given the execution up to and including point τ, with constant probability, L.desc $= \omega_p$ at or before point $t_{2^\alpha+1}$.*

We prove Lemma 4.35 in Section 4.3.4.

Lemma 4.36. *A* Lock() *method call comprises a sequence of 'attempts', where each attempt involves at most a constant number of non-thunk steps. If the point contention of the* Lock() *call at the beginning of an attempt is k, then the attempt fails immediately with probability at most $1 - 1/(2k)$. If the attempt does not fail immediately, then there is a constant probability that the call finishes after at most $k + 2$ additional attempts.*

Proof. Suppose that process p executes a Lock(ω_p) call. Each attempt begins when p executes line 50 (so an attempt corresponds to a loop iteration). At this point, it sets its local variable μ to $\alpha = \lfloor \log C \rfloor$, which is at most $\log k$, since each process adds 1 to C in its Lock() call. The attempt does not fail immediately in the case that p writes to $R[\alpha]$ in line 51. This write occurs with probability $2^{-\alpha-1} \geq 2^{-(\log k)-1} = 1/(2k)$.

Suppose that process p's write succeeds at point τ. From Lemma 4.35, with constant probability there is a point $\tau^\star$ at which L.desc $= \omega_p$, and the number of successful CAS operations on L in

interval $(\tau, \tau^\star]$ is at most $2^\alpha + 1$. (Since p does not exit its loop until ω_p is done, and since no process executes ω_p's thunk until after the first point when L.desc $= \omega_p$, it follows that point $\tau^\star$ is before p's response.) Consider the process q with the first successful CAS on L after point $\tau^\star$. Since q's CAS successfully changes L from ω_p, it takes place after q executes ω_p in line 36, and therefore ω_p is done by the point of q's CAS. It follows that ω_p is done within $2^\alpha + 2$ successful CAS operations on L after point τ.

From Lemma 4.41 (proven in Section 4.3.3), it follows that in each of p's loop iterations, either the pair stored in L changes (via a successful CAS), or p exits the loop at the end of the iteration. It follows that, if p completes $2^\alpha + 2$ loop iterations after point τ, then there are at least $2^\alpha + 2$ successful CAS operations on L by the end of the last such iteration. Hence, with constant probability ω_p is done by that point.

Therefore, with constant probability ω_p is done within $2^\alpha + 2 \le k + 2$ loop iterations of an attempt that does not fail immediately. □

Theorem 4.37. *In Algorithm 3, the expected number of steps of a* `Lock()` *call is in* $O(\mathbf{E}[\kappa] \cdot T)$*, where* κ *is the maximum point contention during the call, and* T *is the maximum step complexity of a thunk.*

Proof. Suppose that process p executes a `Lock()` call, and use the same definition of attempts as in Lemma 4.36. Let T be the random variable counting the total number of loop iterations of the `Lock()` call, before the call responds. Call the *critical point* the point in each loop iteration of the `Lock()` call when p executes the first shared memory step after the random decision in line 51. (That is, at a critical point, either p writes to R, or p reads L in line 35 after deciding not to write to R). We define sequences of points r_j, κ_j, δ_j as follows. Point r_1 is the critical point of the first loop iteration. For $j \ge 1$ such that $r_j < \infty$,

- κ_j is the maximum point contention of p's `Lock()` call until point r_j,
- $\delta_j = \kappa_j + 2$ if p writes to R at point r_j, or $\delta_j = 1$ otherwise, and
- r_{j+1} is the δ_j-th critical point in (r_j, ∞), or $r_{j+1} = \infty$ if no such point exists.

Let J be the largest value of j such that $r_j < \infty$; we end the sequences at r_{J+1}, κ_J, and δ_J.

Intuitively, point r_j corresponds to an of an application of Lemma 4.36. If the attempt containing point r_j fails immediately, then $\delta_1 = 1$, and $r_{j+1} = r_j + 1$. Otherwise, p writes to R at point r_j, and from Lemma 4.36 there is a constant probability γ that p completes at most $k + 2$ iterations after point r_j. In that case, there are fewer than $k + 2$ critical points after r_j, and so $j = J$.

For $j \ge 1$, let Y_j be the random variable defined as

$$Y_j = \begin{cases} \delta_j, & \text{if } j < J \\ \delta_j - \frac{4}{\gamma} \cdot \kappa_j, & \text{if } j = J \\ 0, & \text{if } j > J. \end{cases}$$

For $j \ge 0$, we also define

$$X_j = \sum_{1 \le i \le j} Y_i.$$

We show that the sequence $X_0, X_1, \ldots$ is a supermartingale. Recall from Lemma 4.36 that the probability of an attempt to write is at least $\frac{1}{2k}$. For $j \geq 1$,

$$\begin{aligned}\mathbf{E}\left[Y_j \;\middle|\; j \leq J, \kappa_j = k, Y_1, \ldots, Y_{j-1}\right] &\leq 1 \cdot \left(1 - \frac{1}{2k}\right) + \left(k + 2 - \gamma \cdot \frac{4}{\gamma} k\right) \frac{1}{2k} \\ &= 1 - \frac{1}{2k} + \frac{1}{2} + \frac{1}{k} - 2 \\ &= \frac{1}{2k} - \frac{1}{2} \\ &\leq 0,\end{aligned}$$

where in the last line we use that p contributes one to the point contention, and so $k \geq 1$. Also, since $Y_j = 0$ for $j > J$, we trivially obtain that

$$\mathbf{E}\left[Y_j \;\middle|\; j > J, Y_1, \ldots, Y_{j-1}\right] = 0,$$

and so

$$\mathbf{E}\left[Y_j \;\middle|\; Y_1, \ldots, Y_{j-1}\right] \leq 0.$$

Note also that J is a stopping time. For all j, $\left|Y_j\right| \leq \frac{4}{\gamma} \cdot n$. By repeated applications of Lemma 4.36, for each $j \geq 1$, there is a probability of at least $\frac{\gamma}{2n}$ that $j = J$. Thus, $\mathbf{E}[J] \leq \frac{2n}{\gamma}$. We can therefore use the optional stopping theorem to obtain that

$$\mathbf{E}\left[X_J\right] \leq X_0 = 0.$$

Observe that the definition of δ_j implies that

$$\sum_{1 \leq j \leq J} \delta_j \geq T,$$

since all of p's critical points count towards some δ_j. We thus have

$$X_j \geq T - \frac{4}{\gamma}\kappa_J \geq T - \frac{4}{\gamma}\kappa,$$

where κ is the maximum contention of the `Lock()` call. It follows that

$$\begin{aligned}\mathbf{E}[T] - \frac{4}{\gamma}\mathbf{E}[\kappa] \leq \mathbf{E}\left[X_j\right] &\leq 0 \\ \mathbf{E}[T] &\leq \frac{4}{\gamma}\mathbf{E}[\kappa] \in O(\mathbf{E}[\kappa]).\end{aligned}$$

□

4.3.3 Deterministic Lemmas. In this section we prove several deterministic lemmas used in other step complexity proofs.

Observation 4.38. *If a successful CAS on L takes place at point t, and L.desc $= \omega$ immediately before point t, then ω is done throughout the interval $[t, \infty)$.*

Proof. Suppose that p executes a successful CAS on L in line 47 at point t from (ω, γ). After p executes ω's thunk in line 36, ω is done for the rest of the execution, including point t. □

Lemma 4.39. *If a successful CAS on L takes place at point t, and L.desc $= \omega$ immediately after point t, then ω is not done at point t.*

Proof. Suppose that p executes a successful CAS on L in line 47 at point t from (ω_0, γ_0) to (ω, γ). Let t_0 be the point when p reads L in line 35 as part of the `HelpAndPromote()` call that ends at point t. Since p does not respond in line 41, it follows from lines 37, 39, and 40 that it sets its local variable $w' = \omega$ to some descriptor that is not done at some point after p's execution of line 36. That is, there is some point $t^\star \in (t_0, t)$ at which ω is not done.

Now suppose for the purpose of proving a contradiction that ω.`isDone()` returns `True` at point t, and thus that there is a point $t^\dagger \in (t^\star, t)$ at which ω's thunk responds for the first time. Without loss of generality, further suppose that t is the first point in the execution in which a successful CAS on L sets L.desc to a descriptor that is done.

Since ω can only be done after some process reads ω from L.desc, it follows that there is a point $t' < t^\dagger$ at which L.desc $= \omega$. If $t' < t_0$, then by Observation 4.38, ω is done before L.desc changes from ω, which in turn is before t_0 (when L.desc $= \omega_0$), which is a contradiction; hence, $t' > t_0$. Since L.desc $= \omega_0$ at point t_0 but not at point $t' > t_0$, it follows from Observation 4.38 that ω_0 is done by point t'. But, because L.desc $= \omega_0$ immediately before point t, there is a point after t' and before t at which L.desc changes back to ω_0 while ω_0 is done. Since L can only change via a successful CAS, this contradicts that t is the first point in the execution in which a successful CAS on L sets L.desc to a descriptor that is done. □

Lemma 4.40. *If a successful CAS takes place on L at point t, then at point t, L.*desc *changes to a descriptor that it never stored before point t.*

Proof. Let ω_0 be the dummy descriptor originally stored in L.desc, and, for $i \geq 1$, let ω_i be the value stored in L.desc immediately after the i-th successful CAS on L. Suppose that point t is the j-th successful CAS on L in the execution. Then by applying Observation 4.38 to all successful CAS operations on L up to and including point t, we have that at point t, ω_i is done for all $i < j$. Applying Lemma 4.39 to the CAS at point t, ω_j is not done at point t. Thus, $\omega_j \neq \omega_i$ for all $i \neq j$. □

Note that Lemma 4.40 proves not only that L.desc and by extension L is ABA-free, but also that every successful CAS on L changes L.desc to a distinct value. Since a CAS on L succeeds if no other CAS succeeds on L between the read in line 35 and the CAS attempt in line 47, read and CAS operations on L behave similarly to load-linked/store-conditional operations.

Lemma 4.41. *Suppose that a process p executes a* `HelpAndPromote(`ω`)` *method call that begins at point t_1 and ends at point t_2. If the pair stored in L does not change in the interval $[t_1, t_2]$, then throughout $[t_2, \infty)$, ω is done, and hence p breaks out of its loop when it next executes line 53.*

Proof. Since we assume that L does not change throughout $[t_1, t_2]$, then, by Lemma 4.40, p exits early in line 41 (otherwise p would successfully change L in line 47). Therefore, p's if-condition in line 40 is false, meaning that ω is done when p executes that line. □

4.3.4 *Proof of Lemma 4.35.*

Definition 4.42. *For $1 \leq i \leq 2^\alpha + 1$, let*

- *t_i be the point of the i-th successful CAS on L after point τ,*
- *p_i be the process that executes the CAS at point t_i,*
- *λ_i be the value that p_i assigns to variable λ when it last executes line 42 prior to t_i, and*
- *ℓ_i be the value of L.*level *right before point t_i.*

Let $i^\star$ be the smallest $i \in \{1, \ldots, 2^\alpha\}$ for which $\lambda_i \in \{\alpha - 1, \alpha, \alpha + 1\}$, or ∞ if no such i exists. Let $j^\star$ be $\min\{i^\star, 2^\alpha\}$.

From Lemma 4.41, for each `HelpAndPromote()` call that p executes, either some CAS on L is successful, or p breaks out of its loop after the call. If p completes at least $2^\alpha + 1$ `HelpAndPromote()` calls after point τ before responding from its `Lock()` call, then the above points are all well-defined. Otherwise, if the adversary schedules p for sufficiently many steps, then Lemma 4.35 follows from p responding from the `Lock()` call before all the points take place. Thus, we can assume without loss of generality that all the points in Definition 4.42 are well-defined.

Lemma 4.43 (Bounded Write Attempts). *There is a constant c such that the total number of times that line 51 is executed with $\mu = \alpha$ in the interval $(\tau, t_{j^\star+1})$ is less than $c \cdot 2^\alpha$.*

We prove Lemma 4.43 in Section 4.3.5.

Definition 4.44. *We define the following three "good" events:*

- *E_1 is the event that $\ell_{i^\star} = \alpha$ or $i^\star = \infty$.*
- *E_2 is the event that $\ell_i = \alpha$ for one of the first 2^α values of i that are not $i^\star$, i.e., $i \in \{1, \ldots, 2^\alpha + 1_{i^\star \le 2^\alpha}\}$.*
- *E_3 is the event that the first $c \cdot 2^\alpha$ times that a process executes line 51 after point τ with $\mu = \alpha$ (with the constant c from Lemma 4.43), the process does not write.*

Lemma 4.45. *If E_1, E_2, and E_3 all hold, then $L.\text{desc} = \omega_p$ at some point at or before $t_{2^\alpha+1}$.*

Proof. From events E_1 and E_2, it follows that there is some $h \in \{1, \ldots, j^\star\}$ such that $\ell_h = \alpha$. Then, process p_{h+1} reads descriptor ω_{h+1} from register $R[\alpha]$ at the point $r_{h+1} < t_{h+1}$ when p_{h+1} reads $R[\alpha]$ in line 38. Note that $r_{h+1} > t_h$, otherwise, by Lemma 4.40, the old value used for p_{h+1}'s CAS on L at point t_{h+1} would not match the value of L at that point, since p_{h+1}'s read would happen before t_h.

Event E_3 together with Lemma 4.43 gives that $R[\alpha]$ is never written in the interval $(\tau, t_{j^\star+1}) \supseteq (\tau, t_{h+1}) \supset (\tau, r_{h+1})$. Therefore, the value of register $R[\alpha]$ at point r_{h+1} is $\omega_{h+1} = \omega_p$.

If ω_p is done by the time p_{h+1} reads it in line 39, then there is a point before t_{h+1} at which $L.\text{desc} = \omega_p$. Otherwise, p_{h+1} sets its local variable w' to ω_p, and so $L.\text{desc} = \omega_p$ at point t_{h+1}. □

Lemma 4.46. *Events E_1, E_2, and E_3 are mutually independent, and the probability that all three occur given the weak adaptive adversary is bounded below by a constant.*

Proof. First, observe that each of the three events is defined based on the outcomes of random decisions, and that they do not overlap in any random decisions. Also observe that for each random decision, the weak adaptive adversary knows, immediately before scheduling it, whether it is a random decision that is part of the definition of some E_x, and for which $x \in \{1, 2, 3\}$. In particular, for E_1 and E_2, the adversary knows each process's value of λ before the random decision of ℓ_i, and it knows whether the CAS will be successful, which also implies knowing whether i will be $i^\star$ or not. For E_3, the adversary knows the value of μ before the random decision of whether to write in line 51. However, the adversary does not learn about the random outcomes involved (the values of ℓ_i or whether writes to $R[\alpha]$ take place) until after the random event contributes to that E_x. Therefore, the adversary has no influence on the probabilities of the random decisions involved in E_1, E_2, and E_3. (By contrast, a strong adaptive adversary could let the random decision take place, and based on the outcome decide whether to commit that decision to shared memory, and hence which random decisions contribute to each E_x.)

If $i^\star \le 2^\alpha$, then $\lambda_{i^\star} \in \{\alpha - 1, \alpha, \alpha + 1\}$, and $\alpha \in \{0, \ldots, \lfloor \log n \rfloor\}$. Therefore, with probability $1/2$, process $p_{i^\star}$ executes line 44, in which case with probability at least $1/3$ it sets $\ell_{i^\star}$ to α. Thus, the probability of E_1 is at least $1/6$.

For each ℓ_i, with probability $1/2$ process p_i executes line 46, in which case with probability $2^{-\alpha-1}$ it sets ℓ_i to α. Thus, the probability of E_2 is at least

$$1-(1-1/2\cdot 2^{-\alpha-1})^{2^\alpha} = 1-(1-2^{-\alpha-2})^{2^{\alpha+2}\cdot 2^{-2}} \geq 1-e^{-1/4}.$$

The probability of E_3 is

$$(1-2^{-\alpha-1})^{c\cdot 2^\alpha} \geq \left(4^{-2^{-\alpha-1}}\right)^{c\cdot 2^\alpha} = 2^{-c}. \qquad \square$$

Lemmas 4.46 and 4.45 together imply Lemma 4.35.

4.3.5 *Proof of Lemma 4.43.*

Definition 4.47.

- *For $i \geq 1$, let t_i denote the point of the i-th successful L.*CAS() *operation after τ, as in Definition 4.42.*
- *We say round j, for $j \geq 1$, takes place during interval $I_j = (t_{j-1}, t_j]$, where we define t_0 as point τ.*
- *Round $j \geq 1$ is* unsafe *is there is a point $T_j \in I_j$ at which $\lfloor \log C \rfloor = \alpha$, and otherwise it is* safe.
- *A process is* happy *at point t if it has a pending* Lock() *call at point t, and at some point before t its descriptor is stored in L.*desc. *A process with a pending* Lock() *call at point t that is not happy is* waiting.

Observation 4.48. *Each process can read C in line 50 at most once per round.*

Proof. This follows from the fact that processes call HelpAndPromote() in between each execution of line 50, during which by Lemma 4.41 either L changes (advancing the round) or the process breaks out of its loop. □

Lemma 4.49. *If round j is safe, with $1 \leq j < j^\star$, then no process assigns value α to μ in line 50 during round j.*

Proof. We prove the contrapositive: suppose that a process assigns value α to μ in line 50 during round j. Then at that point it reads a value of $C \in \{2^\alpha, \ldots, 2^{\alpha+1}-1\}$, and therefore the round is unsafe. □

Observation 4.50. *There is no round j with $1 \leq j < j^\star$ such that $2^{\alpha-1} \leq C < 2^{\alpha+2}$ throughout I_j.*

Proof. If there were such a round j, then λ_j would be in $\{\alpha-1, \alpha, \alpha+1\}$, so $j \geq i^\star$. This contradicts that $j < j^\star \leq i^\star$. □

Lemma 4.51. *Consider any two unsafe rounds j_1 and j_2 such that $1 \leq j_1 < j_2 < j^\star$ **and** $j_1 + 1 \neq j_2$ (i.e., j_1 and j_2 are not consecutive). Then in the interval (T_{j_1}, T_{j_2}), more than $2^{\alpha-1}$ C.*FAD() *operations are executed.*

Proof. At points T_{j_1} and T_{j_2}, $2^\alpha \leq C < 2^{\alpha+1}$. By Observation 4.50, there is a point $T \in I_{j_1+1}$ at which either $C < 2^{\alpha-1}$ or $C \geq 2^{\alpha+2}$. In the first case, there are more than $2^\alpha - 2^{\alpha-1} = 2^{\alpha-1}$ C.FAD() operations in (T_{j_1}, T). In the second case, there are more than $2^{\alpha+2} - 2^{\alpha+1} > 2^{\alpha-1}$ C.FAD() operations in (T, T_{j_2}). □

Lemma 4.52. *If round j is unsafe, with $1 \leq j < j^\star$, then in $[T_j, t_{j^\star-1}]$, fewer than $3\cdot 2^\alpha$ C.*FAD() *operations are executed.*

Proof. Observe that only happy processes can decrement C. At point T_j, $C < 2^{\alpha+1}$, and so there are fewer than $2^{\alpha+1}$ happy processes. At the end of each round, at most one process can become happy, and $j^\star - 1 < 2^\alpha$. Thus, fewer than $2^{\alpha+1} + 2^\alpha = 3 \cdot 2^\alpha$ decrements of C can take place in the interval. □

Lemma 4.53. *There are at most* 12 *unsafe rounds among rounds* $\{1, \ldots, j^\star - 1\}$.

Proof. Suppose for the purpose of proving a contradiction that there are 13 such rounds. Then there are at least 7 unsafe rounds, such that no two of them are consecutive, and all are before round $j^\star$. Let j be the first of those 7 unsafe rounds, and j' the last. Let F be the number of C.FAD() operations executed in $(T_j, T_{j'})$. By Lemma 4.52, $F < 3 \cdot 2^\alpha$. By repeated applications of Lemma 4.51, $F > 6 \cdot 2^{\alpha-1} = 3 \cdot 2^\alpha$, which is a contradiction. □

Lemma 4.54. *Suppose that at point t, a waiting process reads C in line 50, and at point t' it decrements C. Then a successful CAS takes place on L in (t, t').*

Proof. Suppose that a process q is waiting at point t, as part of a Lock(ω_q) call. Since q is waiting at point t, at all points up to t, L.desc $\neq \omega_q$. Recall that L.desc can only change with a successful CAS on L, and that ω_q can only be done after a point when L.desc $= \omega_q$. It thus follows that ω_q is not done until at least the next successful CAS on L after t. Since q does not exit its loop in line 53 to decrement C in line 54 until after ω_q is done, the lemma follows. □

Lemma 4.55. *In any round, fewer than $2^{\alpha+1}$ waiting processes assign value α to μ in line 50.*

Proof. Let Y be the set of waiting processes that execute line 50 during a given round. Then each process in Y increments C before or during that round, but, by Lemma 4.54, does not decrement C before the end of the round. Hence, the last process in Y to execute line 50 reads a value of at least $|Y|$ from C. Thus, $|Y| < 2^{\alpha+1}$. □

Lemma 4.56. *Throughout $(\tau, t_{j^\star+1})$, fewer than $14 \cdot 2^{\alpha+1}$ executions of line 50 that assign α to μ are from waiting processes.*

Proof. From Lemma 4.49, no process assigns α to μ in line 50 in a safe round that is before round $j^\star$. From Lemma 4.53, there are at most 12 unsafe rounds among rounds $\{1, \ldots, j^\star - 1\}$. Thus, there are at most 14 rounds among rounds $\{1, \ldots, j^\star + 1\}$ in which any process can assign α to μ in line 50. From Observation 4.48 and Lemma 4.55, fewer than $2^{\alpha+1}$ executions of line 50 which assign α to μ can take place in any given round from waiting processes. □

Lemma 4.57. *Throughout $(\tau, t_{j^\star+1})$, fewer than $3 \cdot 2^\alpha$ executions of line 50 that assign α to μ are from happy processes.*

Proof. Observe that each process executes line 50 at most once after becoming happy and before responding from its Lock() call.

Consider the set Y of processes which are happy at point τ and assign α to μ in line 50 at some point in $(\tau, t_{j^\star+1})$. Let $t^\star$ be the first point after τ at which a process in Y reads C in line 50. Since no process in Y responds before point $t^\star$, it follows that $C \geq |Y|$ at point $t^\star$. Since we also have that $C < 2^{\alpha+1}$ at point $t^\star$, it follows that $|Y| \leq C < 2^{\alpha+1}$.

The remaining set of happy processes that assign α to μ in $(\tau, t_{j^\star+1})$ become happy after point τ. At most one process becomes happy at the point of each successful CAS on L, and therefore at most $j^\star \leq 2^\alpha$ processes can become happy in $(\tau, t_{j^\star+1})$. □

Lemma 4.58. *At point τ, fewer than $2^{\alpha+1}$ processes are poised to execute line 51 with $\mu = \alpha$ as their next step.*

Proof. Let Y be the set of processes that, at point τ, are poised to execute line 51 with $\mu = \alpha$ as their next step. Observe that each process in Y has not yet decremented C at point τ, but has incremented C before point τ. Consider the last point $t^\star < \tau$ at which a process in Y reads C in line 50. This process reads a value smaller than $2^{\alpha+1}$ from C. Since $t^\star$ is after each process in Y increments C, and before any of them decrement C, it follows that $|Y| \leq C$ at point $t^\star$, and hence that $|Y| < 2^{\alpha+1}$. □

We can now prove Lemma 4.43 by bounding the number of executions of line 51 with $\mu = \alpha$ in the interval $(\tau, t_{j^\star+1})$. Each such execution is either from a process that previously assigns α to μ in line 50 in the interval $(\tau, t_{j^\star+1})$, or a process already poised to execute line 51 with $\mu = \alpha$ at point τ. From Lemma 4.56, fewer than $14 \cdot 2^{\alpha+1}$ executions of line 50 in the interval $(\tau, t_{j^\star+1})$ that assign α to μ are from waiting processes. From Lemma 4.57, fewer than $3 \cdot 2^\alpha$ executions of line 50 in the interval $(\tau, t_{j^\star+1})$ are from happy processes. Finally, from Lemma 4.58, fewer than $2^{\alpha+1}$ processes are poised to execute line 51 with $\mu = \alpha$ at point τ.

5 Conclusion

In this paper, we explored various randomized locks with sublinear space complexity that adapt to point contention. Our most space efficient algorithm uses only three shared variables, has only slightly higher than optimal step complexity, and is easy to implement. We also showed how to achieve optimal expected step complexity using $O(\log n)$ shared variables. It remains an open problem whether both optimal space and step complexity can be achieved simultaneously. Two of our algorithms work against the weak adaptive adversary, and one against the oblivious adversary. We do not know if an algorithm with similar space and step complexity exists in the strong adaptive adversary model.

Acknowledgments

Natural Sciences and Engineering Research Council of Canada
Conseil de recherches en sciences naturelles et en génie du Canada
Canada

This work was supported by the Natural Sciences and Engineering Research Council of Canada (NSERC), funding reference numbers RGPIN-2025-04233 and CGSD-578918-2023/198, as well as the University of Calgary Eyes High Doctoral Recruitment Scholarship program.

Cette recherche a été financée par le Conseil de recherches en sciences naturelles et en génie du Canada (CRSNG), numéros de référence RGPIN-2025-04233 et CGSD-578918-2023/198.

References

[1] Kahbod Aeini. 2025. *A Space-Optimal Randomized Wait-Free Lock.* M.Sc. thesis. University of Calgary, Calgary, Canada. https://hdl.handle.net/1880/123667

[2] Yehuda Afek, Dalia Dauber, and Dan Touitou. 1995. Wait-free made fast. In *Proceedings of the 27th Annual ACM Symposium on Theory of Computing (STOC)*. 538–547. https://doi.org/10.1145/225058.225271

[3] James Aspnes and Maurice Herlihy. 1990. Fast Randomized Consensus Using Shared Memory. *Journal of Algorithms* 11, 3 (1990), 441–461.

[4] Greg Barnes. 1993. A Method for Implementing Lock-Free Shared-Data Structures. In *Proceedings of the 5th ACM Symposium on Parallel Algorithms and Architectures (SPAA)*. 261–270. https://doi.org/10.1145/165231.165265

[5] Naama Ben-David and Guy E. Blelloch. 2025. Fast and fair randomized wait-free locks. *Distributed Computing* 38, 1 (2025), 51–72. https://doi.org/10.1007/S00446-024-00474-4

[6] Naama Ben-David, Guy E. Blelloch, and Yuanhao Wei. 2022. Lock-free locks revisited. In *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPOPP)*. 278–293. https://doi.org/10.1145/3503221.3508433

[7] Jeremy Brown, J. P. Grossman, and Tom Knight. 2002. A lightweight idempotent messaging protocol for faulty networks. In *Proceedings of the 14th ACM Symposium on Parallel Algorithms and Architectures (SPAA)*. 248–257. https://doi.org/10.1145/564870.564912

[8] Marc de Kruijf and Karthikeyan Sankaralingam. 2013. Idempotent code generation: Implementation, analysis, and evaluation. In *Proceedings of the 2013 IEEE/ACM International Symposium on Code Generation and Optimization, CGO*. 27:1–27:12. https://doi.org/10.1109/CGO.2013.6495002

[9] Marc de Kruijf, Karthikeyan Sankaralingam, and Somesh Jha. 2012. Static analysis and compiler design for idempotent processing. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI*. 475–486. https://doi.org/10.1145/2254064.2254120

[10] Edsger W. Dijkstra. 1965. Solution of a problem in concurrent programming control. *Communications of the ACM* 8, 9 (1965), 569. https://doi.org/10.1145/365559.365617

[11] Faith Ellen Fich, Maurice Herlihy, and Nir Shavit. 1998. On the Space Complexity of Randomized Synchronization. *Journal of the ACM* 45, 5 (1998), 843–862. https://doi.org/10.1145/290179.290183

[12] Wojciech Golab, Lisa Higham, and Philipp Woelfel. 2011. Linearizable implementations do not suffice for randomized distributed computation. In *Proceedings of the 43rd Annual ACM Symposium on Theory of Computing (STOC)*. 373–382. https://doi.org/10.1145/1993636.1993687

[13] Timothy L. Harris, Keir Fraser, and Ian A. Pratt. 2002. A practical multi-word compare-and-swap operation. In *Proceedings of the 16th International Symposium on Distributed Computing (DISC)*. 265–279. https://doi.org/10.1007/3-540-36108-1_18

[14] Maurice Herlihy. 1991. Wait-Free Synchronization. *ACM Transactions on Programming Languages and Systems* 13, 1 (Jan. 1991), 124–149. https://doi.org/10.1145/114005.102808

[15] P. Z. Ingerman. 1961. Thunks: A way of compiling procedure statements with some comments on procedure declarations. *Communications of the ACM* 4, 1 (Jan. 1961), 55–58. https://doi.org/10.1145/366062.366084

[16] Chaitanya Koparkar. 2022. A primer on pointer tagging. *XRDS* 29, 1 (2022), 66–68. https://doi.org/10.1145/3558200

[17] Leslie Lamport. 2019. A new solution of Dijkstra's concurrent programming problem. In *Concurrency: the Works of Leslie Lamport*. ACM, 171–178. https://doi.org/10.1145/3335772.3335782

[18] Maged M. Michael and Michael L. Scott. 1996. Simple, Fast, and Practical Non-Blocking and Blocking Concurrent Queue Algorithms. In *Proceedings of the 15th SIGACT-SIGOPS Symposium on Principles of Distributed Computing (PODC)*. 267–275. https://doi.org/10.1145/248052.248106

[19] Gary L. Peterson. 1981. Myths About the Mutual Exclusion Problem. *Information Processing Letters* 12, 3 (1981), 115–116. https://doi.org/10.1016/0020-0190(81)90106-X

[20] Guy L. Steele and Gerald J. Sussman. 1976. *Lambda: The Ultimate Imperative*. Technical Report. Massachusetts Institute of Technology. http://hdl.handle.net/1721.1/5790

[21] Shahar Timnat and Erez Petrank. 2014. A practical wait-free simulation for lock-free data structures. In *Proceedings of the 19th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPOPP)*. 357–368. https://doi.org/10.1145/2555243.2555261

[22] John Turek, Dennis Shasha, and Sundeep Prakash. 1992. Locking without blocking: Making lock based concurrent data structure algorithms nonblocking. In *Proceedings of the 11th ACM SIGACT-SIGMOD Symposium on Principles of Database Systems*. 212–222. https://doi.org/10.1145/137097.137873

[23] Yuanhao Wei. 2023. *General Techniques for Efficient Concurrent Data Structures*. Ph. D. Dissertation. Carnegie Mellon University. https://doi.org/10.1184/R1/23739501.v1